

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Lab Task

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ikram ul Haq

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

1. Introduction

In this lab I set up a complete CI/CD workflow for a **Laravel Student Management System**. I prepared an Ubuntu server on AWS EC2, installed Jenkins on it, built the Laravel application with student CRUD (create, read, update, delete) functionality, pushed the code to GitHub, and then automated the build, test and deployment steps using Jenkins Freestyle jobs and a Jenkins Pipeline. Finally, I served the application to the public through Nginx.

Every step below follows the same pattern: I first explain what I did and why, then show the command, explain it, describe the result, and only then show the terminal or browser screenshot as evidence. The practical work was done on 20 October 2026 on the EC2 instance `JenkinsLabTask` (public IPv4 address `13.51.206.252`).

2. Preparing the AWS EC2 Server

2.1 Reviewing the EC2 Instance

I started by opening the EC2 console to confirm the details of the server I would be working on. In real DevOps work this is the first check, because the public IP address and the instance state decide how every later service (SSH, Jenkins, the website) will be reached.

The instance summary shows that the instance `JenkinsLabTask` (ID `i-01c3abe1ab17178b7`) is in the **Running** state in the Europe (Stockholm) region. Its public IPv4 address is `13.51.206.252` and its private IPv4 address is `172.31.33.121`. I used this public IP for SSH, Jenkins and the website throughout the lab.

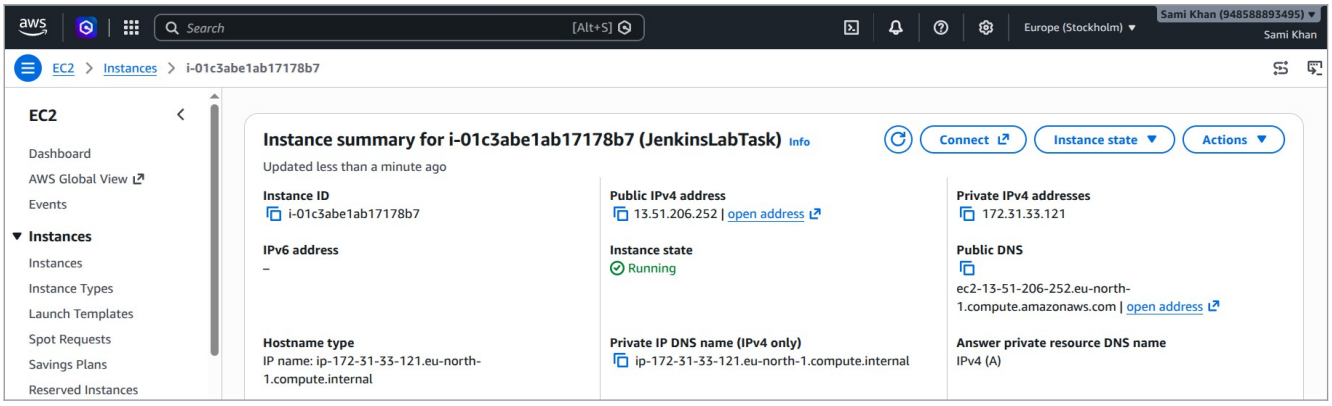


Figure 1: *EC2 instance summary of JenkinsLabTask showing its running state and public IPv4 address.*

2.2 Updating the Server and Installing the Prerequisites

After connecting to the server over SSH as the `ubuntu` user, I refreshed the package index and then installed every package that Jenkins and the Laravel project need: Java (required by Jenkins), Git, Composer, PHP with its extensions, and the MySQL server. Updating first makes sure Ubuntu installs the newest available versions.

Command used

```
sudo apt update
```

Command explanation

- `sudo` runs the command with administrator rights.
- `apt update` downloads the latest list of available packages from the Ubuntu repositories; it does not install anything by itself.

The output shows the Ubuntu `resolute` repositories (InRelease, updates, backports and universe) being checked and downloaded, which means the package list was refreshed successfully.

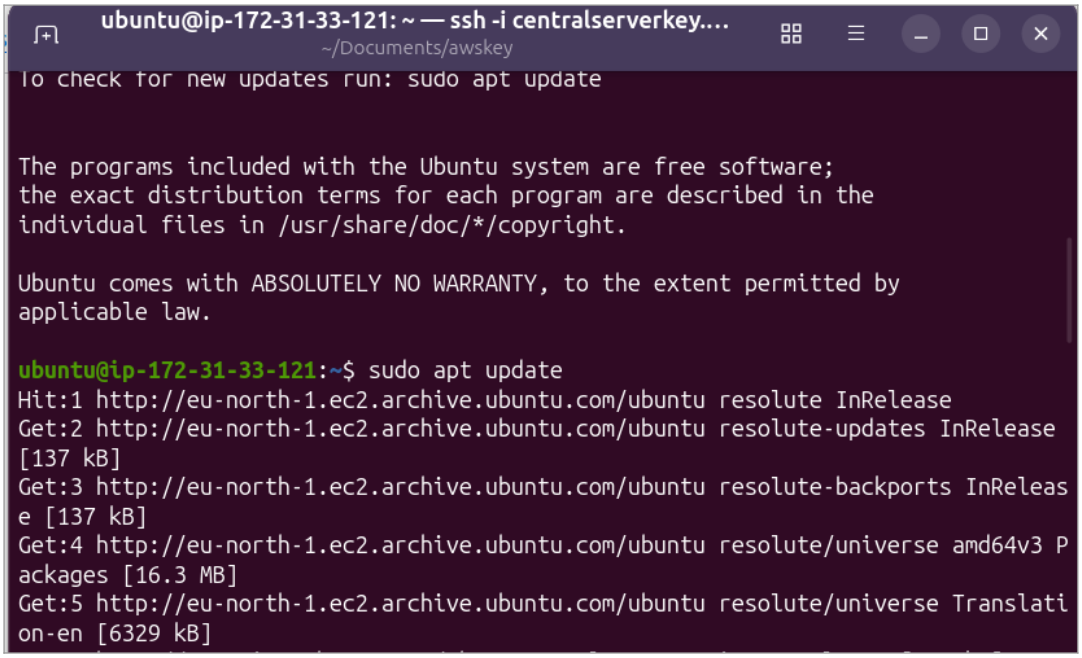


Figure 2: *Running sudo apt update over the SSH session on the EC2 server.*

Next I installed all the required packages with one `apt install` command. The `\` at the end of each line only continues the command on the next line so that it stays readable.

Command used

```
sudo apt install -y \
  fontconfig \
  openjdk-21-jre \
  git \
  curl \
  unzip \
  composer \
  php-cli \
  php-mysql \
  php-xml \
  php-mbstring \
  php-curl \
  php-zip \
  php-bcmath \
  php-intl \
  mysql-server
```

Command explanation

- `apt install` installs the listed packages and their dependencies.
- `-y` answers "yes" automatically to the confirmation prompt.
- `openjdk-21-jre` is the Java runtime that Jenkins needs; `composer`, `php-cli` and the other `php-*` packages provide the PHP environment for Laravel; `mysql-server` provides the database; `git`, `curl` and `unzip` are general tools.

The first screenshot shows the command as I entered it. The second screenshot shows apt's summary: it lists `composer`, `openjdk-21-jre`, `fontconfig`, `mysql-server` and the PHP packages under "Installing", along with their dependencies, and reports that 195 packages can be upgraded.

```
ubuntu@ip-172-31-33-121:~$ sudo apt update

sudo apt install -y \
  fontconfig \
  openjdk-21-jre \
  git \
  curl \
  unzip \
  composer \
  php-cli \
  php-mysql \
  php-xml \
  php-mbstring \
  php-curl \
  php-zip \
  php-bcmath \
  php-intl \
  mysql-server
```

Figure 3: *The apt install command listing the Java, Git, Composer, PHP and MySQL packages.*

```
195 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  curl  git  git-man  libcurl3t64-gnutls  libcurl4t64  libx11-6  libx11-data

Installing:
  composer      openjdk-21-jre  php-curl      php-mysql  unzip
  fontconfig    php-bcmath      php-intl      php-xml
  mysql-server  php-cli         php-mbstring  php-zip

Installing dependencies:
  adwaita-icon-theme          libtcmalloc-minimal4t64
  alsa-topology-conf          libthai-data
  alsa-ucm-conf               libthai0
  at-spi2-common              libvulkan1
  at-spi2-core                libwayland-client0
  bubblewrap                  libwayland-cursor0
  ca-certificates-java        libwayland-egl1
  dconf-gsettings-backend     libx11-xcb1
  dconf-service               libxaw7
  fontconfig-config           libxcb-dri3-0
```

Figure 4: *Apt summary showing the packages and dependencies selected for installation.*

2.3 Verifying the Installed Versions

Once the installation finished, I checked that each tool was really available by asking it for its version. This is a quick way to prove that the environment is ready before moving on to Jenkins.

Command used

```
java -version
php -v
composer --version
git --version
mysql --version
```

Command explanation

- `java -version`, `php -v`, `composer --version`, `git --version` and `mysql --version` each print the installed version of the corresponding tool.

The output confirms OpenJDK 21.0.12.1, PHP 8.5.4 (cli), Composer 2.9.5, Git 2.53.0 and MySQL 8.4.11 on Ubuntu. All five tools respond, so the server meets the prerequisites.

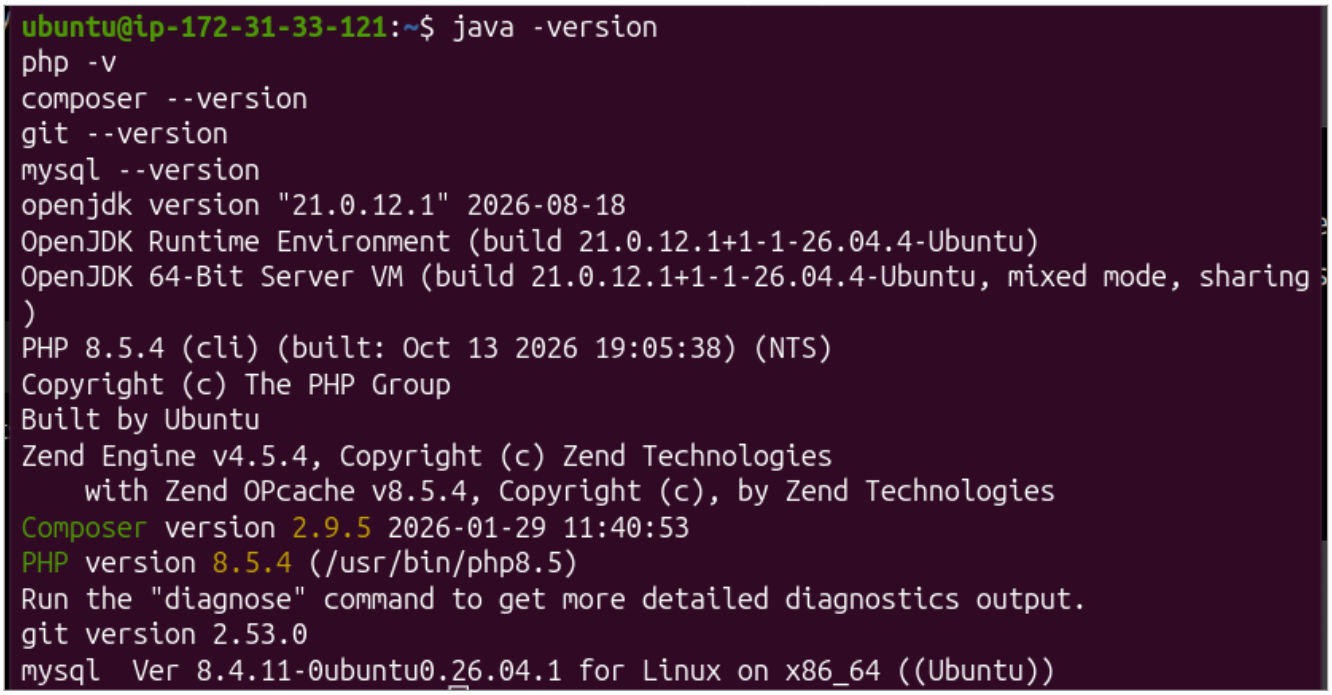


Figure 5: *Version checks for Java, PHP, Composer, Git and MySQL.*

2.4 Checking the MySQL Service

Laravel needs a running database, so I checked the state of the MySQL service that was installed together with the packages above.

Command used

```
sudo systemctl status mysql --no-pager
```

Command explanation

- `systemctl status mysql` shows whether the `mysql` service is running.
- `--no-pager` prints the full result directly in the terminal instead of opening a scrolling viewer.

The output shows `mysql.service` as **enabled** and **active (running)** with the status “Server is operational”, so the database server is ready to be used.

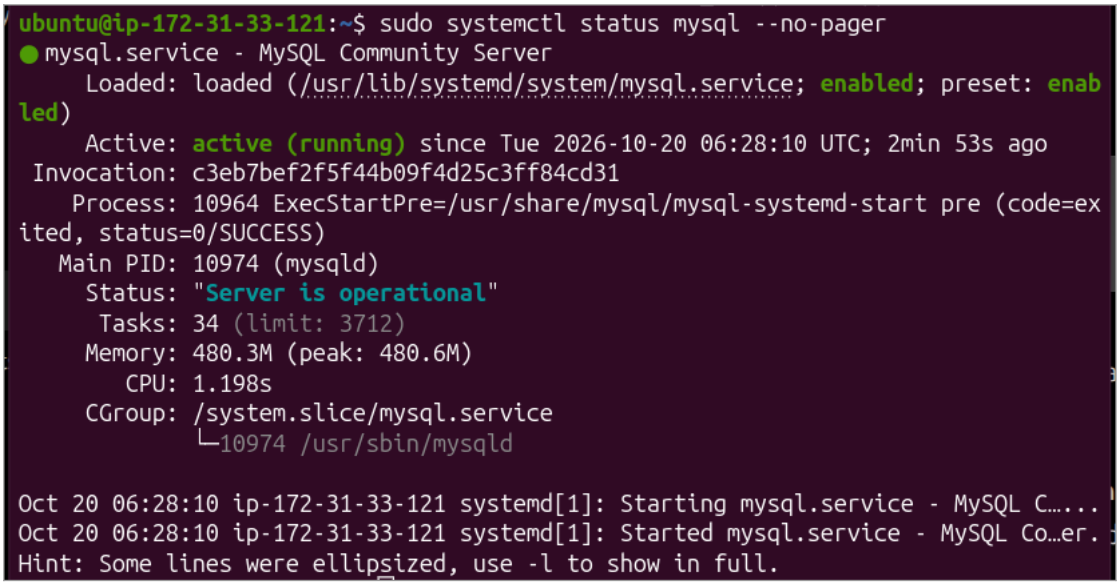


Figure 6: *MySQL service reported as active (running) by systemctl.*

3. Installing and Configuring Jenkins

3.1 Adding the Jenkins Repository

Jenkins is not part of the default Ubuntu package list, so I added the official Jenkins repository first. I created the keyring folder, downloaded the repository signing key, and registered the repository so that `apt` can verify and install Jenkins packages.

Command used

```
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" \
  | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Command explanation

- `mkdir -p /etc/apt/keyrings` creates the folder for repository keys (no error if it already exists).
- `wget -O` downloads the Jenkins signing key and saves it as `/etc/apt/keyrings/jenkins-keyring.asc`.
- `echo "deb [signed-by=...] ..."` writes the repository line, and `sudo tee /etc/apt/sources.list.d/jenkins.list` saves it into apt's sources; `> /dev/null` hides the echoed text.

The output shows `pkg.jenkins.io` being resolved, an "HTTP request sent ... 200 OK" response, and the key (1680 bytes) being saved to `/etc/apt/keyrings/jenkins-keyring.asc`. The repository line was then written to the Jenkins sources file.

```
ubuntu@ip-172-31-33-121:~$ sudo mkdir -p /etc/apt/keyrings
ubuntu@ip-172-31-33-121:~$ sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
--2026-10-20 06:31:56--  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 199.232.174.133, 2a04:4e42:6b::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|199.232.174.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1680 (1.6K) [application/octet-stream]
Saving to: '/etc/apt/keyrings/jenkins-keyring.asc'

/etc/apt/keyrings/je 100%[===================>]   1.64K  --.-KB/s    in 0s

2026-10-20 06:31:56 (41.7 MB/s) - '/etc/apt/keyrings/jenkins-keyring.asc' saved [
1680/1680]

ubuntu@ip-172-31-33-121:~$ echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring
.asc] https://pkg.jenkins.io/debian-stable binary/" \
  | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Figure 7: *Downloading the Jenkins signing key and adding the Jenkins apt repository.*

3.2 Installing Jenkins

With the repository registered, I refreshed the package list again and installed Jenkins.

Command used

```
sudo apt update
sudo apt install -y jenkins
```

Command explanation

- `sudo apt update` reloads the package list so that the new Jenkins repository is included.
- `sudo apt install -y jenkins` installs Jenkins without asking for confirmation.

The first screenshot shows the update, including the `pkg.jenkins.io` repository in the list. The second screenshot shows apt installing `jenkins` together with the dependency `net-tools`.

```
ubuntu@ip-172-31-33-121:~$ sudo apt update
sudo apt install -y jenkins
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InReleas
e
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
```

Figure 8: *Refreshing the package list and starting the Jenkins installation.*

```
188 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  jenkins

Installing dependencies:
  net-tools
```

Figure 9: *Apt installing the jenkins package and its dependency net-tools.*

3.3 Starting Jenkins and Checking Its Status

I then enabled Jenkins so that it starts automatically whenever the server boots, started it immediately, and checked that it was running.

Command used

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins --no-pager
```

Command explanation

- `systemctl enable jenkins` makes the Jenkins service start automatically at boot.
- `systemctl start jenkins` starts the service right now.
- `systemctl status jenkins --no-pager` displays the current state of the service.

The first screenshot shows the enable step completing and the start command being entered. The second shows `jenkins.service` as **enabled** and **active (running)** since 06:33:03 UTC, with a Java process (`/usr/bin/java`) as its main process.

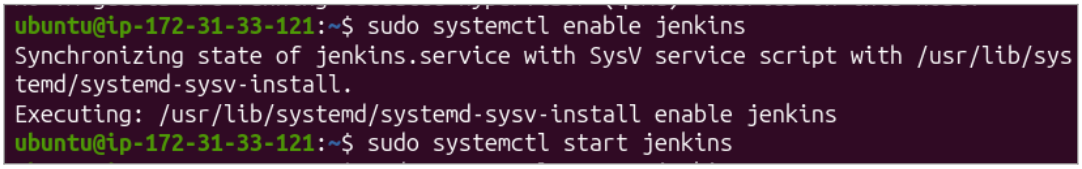


Figure 10: *Enabling and starting the Jenkins service with systemctl.*

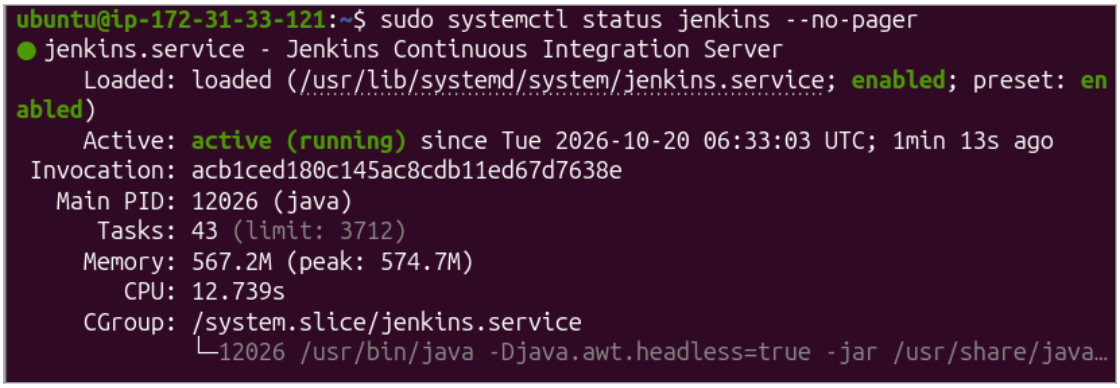


Figure 11: *Jenkins Continuous Integration Server reported as active (running).*

3.4 Retrieving the Initial Admin Password

Jenkins locks its first login with a one-time administrator password that it stores in a file on the server. I displayed that file so that I could unlock Jenkins in the browser.

Command used

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Command explanation

- `cat` prints the contents of a file, and `sudo` is needed because the file is readable only by Jenkins and administrators.

The command printed a 32-character password, which I copied into the Jenkins unlock page in the next steps.

```
ubuntu@ip-172-31-33-121:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
409f8d0e1a4148c9a6d2ca362ea627a5
ubuntu@ip-172-31-33-121:~$
```

Figure 12: *Displaying the Jenkins initial administrator password.*

3.5 Opening Port 8080 in the Security Group

Jenkins listens on port 8080, but AWS blocks all inbound traffic that is not explicitly allowed. I therefore edited the inbound rules of the EC2 Security Group so that the browser could reach Jenkins. This is a configuration step in the AWS console, not a terminal command.

The screenshot shows two inbound rules: **SSH** on TCP port 22 and **Custom TCP** on port **8080**, both limited to my own IP address (`149.40.215.126/32`) through the "My IP" source. Restricting the source keeps Jenkins from being exposed to the whole internet.

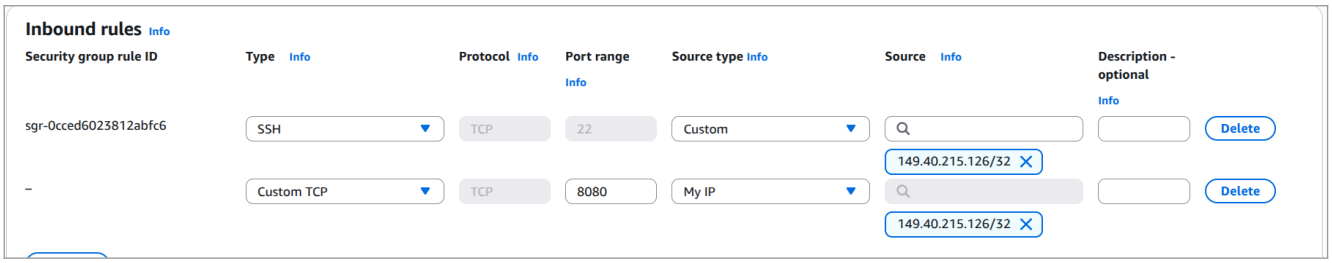


Figure 13: *Security Group inbound rules allowing SSH (22) and Jenkins (8080) from my IP address.*

3.6 Completing the Jenkins Setup Wizard

With port 8080 open, I browsed to `http://13.51.206.252:8080`. The first page, "Unlock Jenkins", asks for the administrator password stored in `/var/lib/jenkins/secrets/initialAdminPassword`, which I had just displayed.

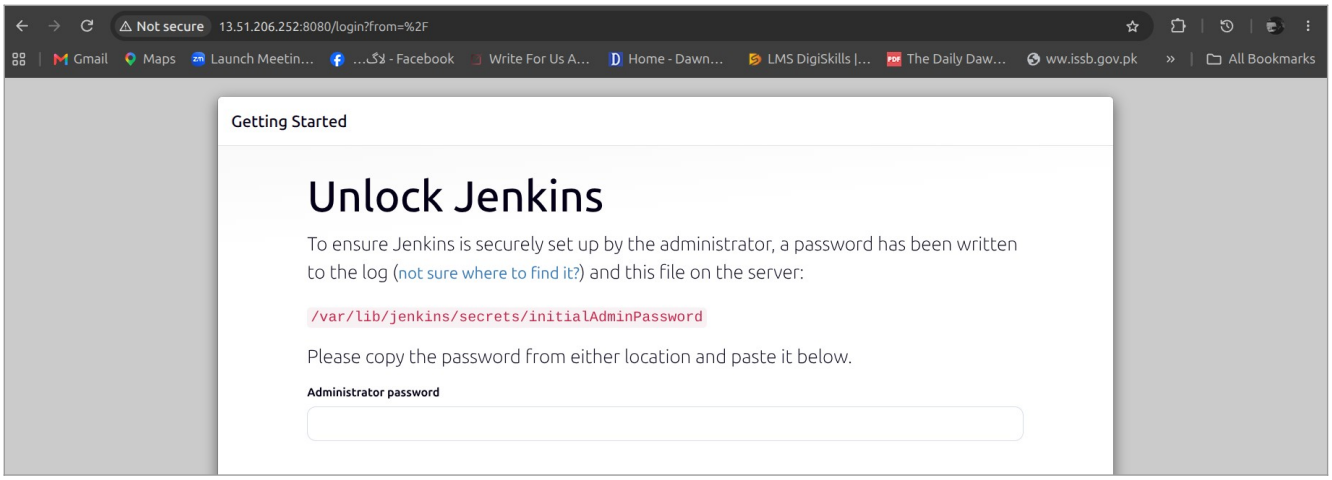


Figure 14: *The Unlock Jenkins page asking for the initial administrator password.*

After unlocking Jenkins, the "Getting Started" screen began installing the suggested plugins. The list includes Pipeline, Git, GitHub Branch Source, Folders, Timestamper, Workspace Cleanup and others that I needed later for Freestyle jobs and pipelines.

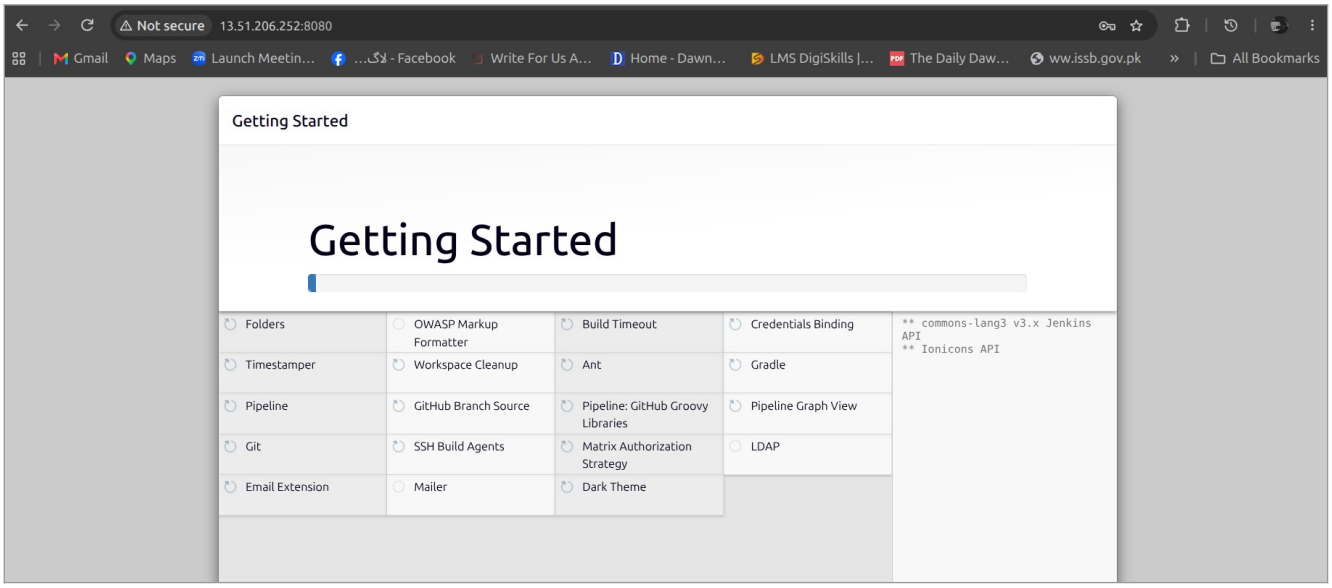


Figure 15: *Jenkins installing the suggested plugins during first-time setup.*

When the plugins finished installing, I created the first administrator account (username `samikhan5055`, full name Sami Khan) so that I would no longer depend on the temporary password. The password fields are hidden in the screenshot.

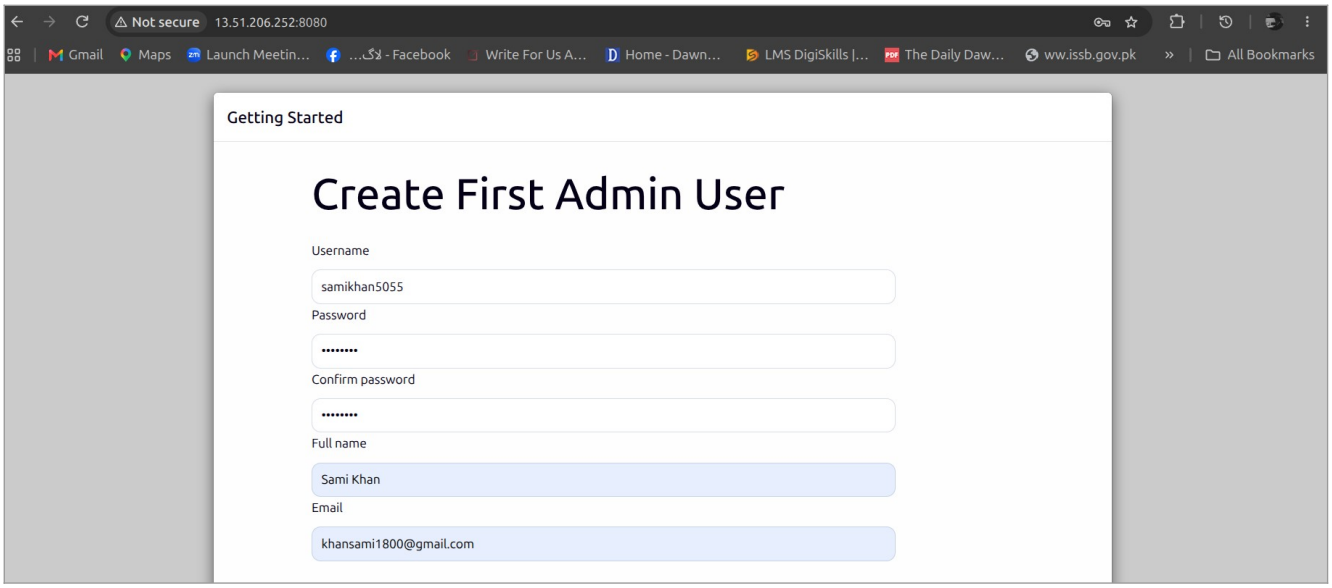


Figure 16: *Creating the first Jenkins administrator user.*

The next page, Instance Configuration, asks for the Jenkins URL that Jenkins uses to build its links. I kept `http://13.51.206.252:8080/`.

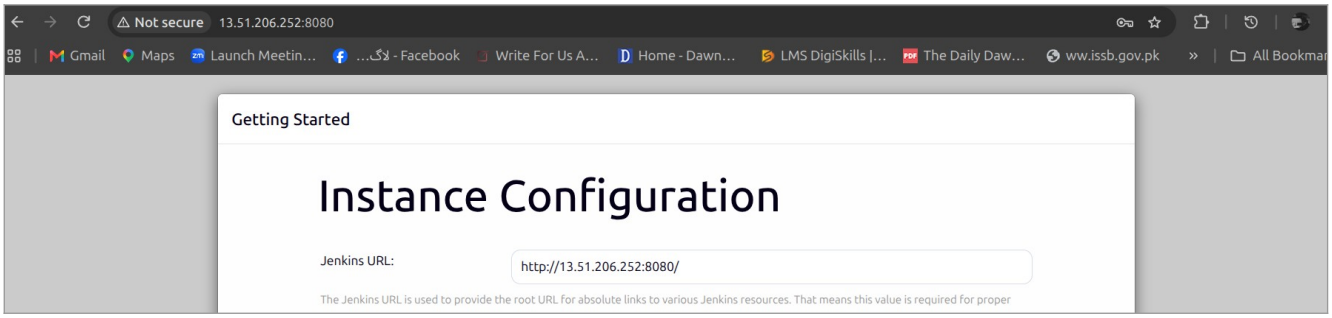


Figure 17: *Instance Configuration page showing the Jenkins URL http://13.51.206.252:8080/.*

Finally, Jenkins opened its dashboard with the message “Welcome to Jenkins!”. The build queue is empty and no jobs exist yet, which confirms that Jenkins is installed and ready for use.

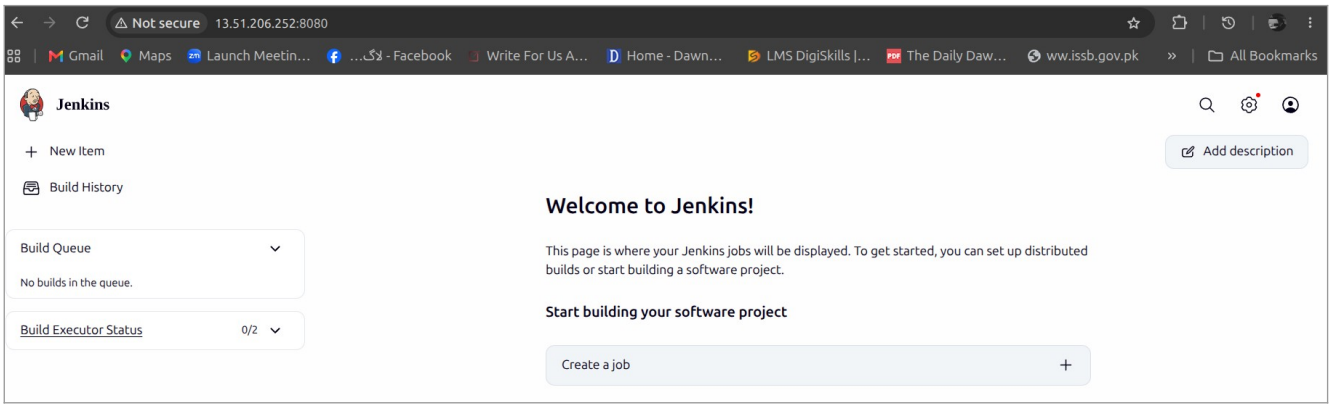


Figure 18: *The Jenkins dashboard after completing the setup wizard.*

4. Setting Up the Laravel Project

4.1 Cloning the Project from GitHub

Next I downloaded the Laravel project from my GitHub repository onto the server. Keeping the code in GitHub is what later allows Jenkins to fetch the same source for its builds.

Command used

```
cd ~
git clone https://github.com/samikhan5055/laravel-project.git
cd laravel-project
```

Command explanation

- `cd ~` moves to the home directory of the `ubuntu` user.
- `git clone <url>` downloads a full copy of the repository, including its history.
- `cd laravel-project` enters the new project folder.

The output shows Git enumerating, counting and compressing 86 objects and receiving them at 100%, followed by “Resolving deltas: 100%”. The prompt now shows `~/laravel-project`, so the clone succeeded.

```
ubuntu@ip-172-31-33-121:~$ cd ~
git clone https://github.com/samikhan5055/laravel-project.git
cd laravel-project
Cloning into 'laravel-project'...
remote: Enumerating objects: 86, done.
remote: Counting objects: 100% (86/86), done.
remote: Compressing objects: 100% (63/63), done.
remote: Total 86 (delta 6), reused 84 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (86/86), 75.32 KiB | 6.85 MiB/s, done.
Resolving deltas: 100% (6/6), done.
ubuntu@ip-172-31-33-121:~/laravel-project$
```

Figure 19: *Cloning the laravel-project repository from GitHub.*

4.2 Creating the MySQL Database and User

The application needs its own database and a dedicated database user instead of the MySQL administrator account. Inside the MySQL client I created the database `student_management`, a user `student_app`, gave that user full rights on this one database only, and reloaded the privileges.

Commands used (inside the MySQL client)

```
CREATE DATABASE student_management;
CREATE USER 'student_app'@'localhost'
    IDENTIFIED BY 'student';
GRANT ALL PRIVILEGES ON student_management.*
    TO 'student_app'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Command explanation

- `CREATE DATABASE` creates an empty database named `student_management`.
- `CREATE USER ... IDENTIFIED BY` creates the user `student_app`, which may connect only from `localhost`.
- `GRANT ALL PRIVILEGES ON student_management.*` lets that user manage every table of this database only.
- `FLUSH PRIVILEGES` makes MySQL reload the permission tables, and `EXIT` leaves the client.

Every statement returned “Query OK” (1 row affected for `CREATE DATABASE`, 0 rows affected for the others), so the database and user were created without errors.

```
mysql> CREATE DATABASE student_management;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> CREATE USER 'student_app'@'localhost'
    -> IDENTIFIED BY 'student';
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> GRANT ALL PRIVILEGES ON student_management.*
    -> TO 'student_app'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> EXIT;
```

Figure 20: *Creating the student_management database and the student_app user in MySQL.*

4.3 Configuring the .env File

Laravel reads its settings from a `.env` file in the project folder. I edited it in the `nano` editor so that the application would use the MySQL database I had just created.

The file shown in the screenshot sets the application name to “Student Management System”, `APP_URL` to `http://13.51.206.252:8001`, and the database settings to `DB_CONNECTION=mysql`, `DB_HOST=127.0.0.1`, `DB_PORT=3306`, `DB_DATABASE=student_management`, `DB_USERNAME=student_app` and `DB_PASSWORD=student`. Sessions and cache use files (`SESSION_DRIVER=file`, `CACHE_STORE=file`) and `QUEUE_CONNECTION=sync`. The title bar shows an asterisk because the file was still being edited when the screenshot was taken.

```
  GNU nano 8.7.1                                .env *
APP_NAME="Student Management System"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://13.51.206.252:8001

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=student_management
DB_USERNAME=student_app
DB_PASSWORD=student

SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync
```

Figure 21: *Editing the Laravel .env file with the MySQL connection settings.*

4.4 Installing the Laravel Dependencies with Composer

Laravel itself and its libraries are not stored in the repository; Composer downloads them according to the `composer.lock` file. I ran Composer inside the project folder.

Command used

```
composer install --no-interaction --prefer-dist
```

Command explanation

- `composer install` installs the exact package versions recorded in `composer.lock`.
- `--no-interaction` stops Composer from asking questions, which is also required when the same command later runs inside Jenkins.
- `--prefer-dist` downloads packaged releases instead of cloning each library, which is faster.

The first screenshot shows “Installing dependencies from lock file” and “Package operations: 110 installs, 0 updates, 0 removals”, with packages such as `doctrine/inflector` and `fakerphp/faker` being downloaded. The second screenshot shows the final step, `php artisan package:discover`, where Laravel registers packages such as `laravel/tinker` and `nesbot/carbon`; each is marked DONE.

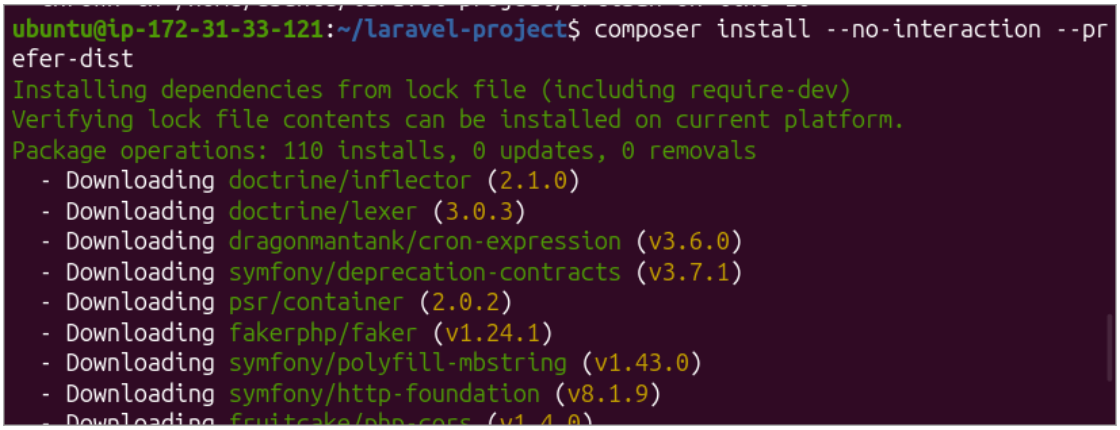


Figure 22: *Composer installing 110 packages from the lock file.*

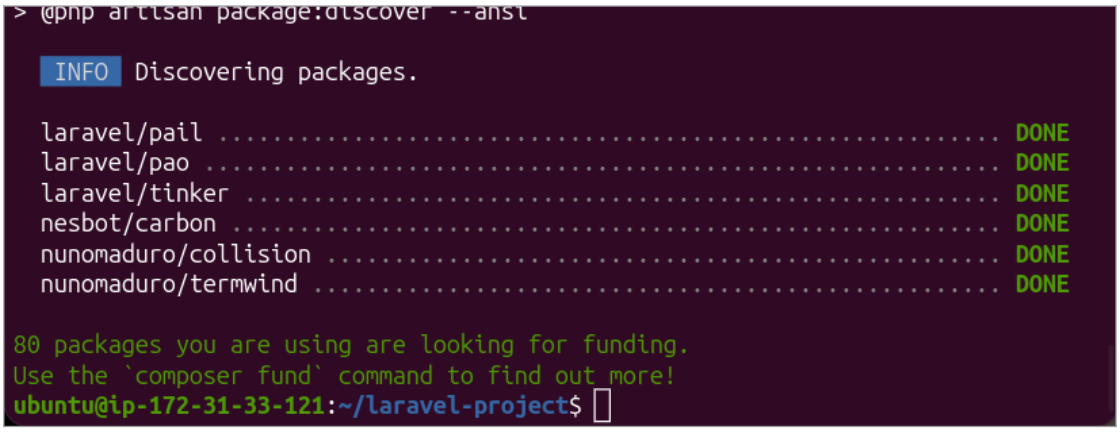


Figure 23: *Laravel package discovery completing after the Composer installation.*

4.5 Running the Default Migrations

Migrations are PHP files that create the database tables. I ran the migrations that come with a fresh Laravel project so that the connection to MySQL was proven before I added my own table.

Command used

```
php artisan migrate
php artisan migrate:status
```

Command explanation

- `php artisan` is Laravel's command-line tool; `migrate` applies all pending migrations to the configured database.
- `migrate:status` lists every migration and shows whether it has already run.

`migrate` prepared the database, created the migration table, and ran the three default migrations (`create_users_table`, `create_cache_table` and `create_jobs_table`), each marked DONE. `migrate:status` then lists the same three migrations in batch [1] with the status **Ran**.



Figure 24: *Running php artisan migrate against the MySQL database.*

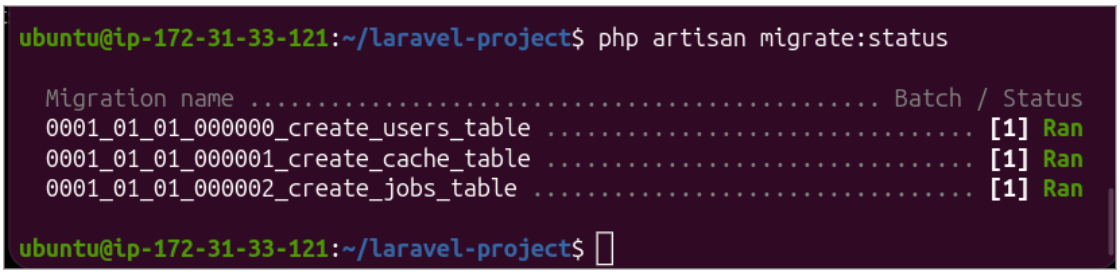


Figure 25: *php artisan migrate:status showing the default migrations as Ran.*

5. Building the Student Management Application

5.1 Generating the Model, Migration and Controller

For the student CRUD feature I used Laravel's generators. One command creates the `Student` model together with a migration file, and another creates a resource controller that contains the standard CRUD methods.

Command used

```
php artisan make:model Student -m
php artisan make:controller StudentController --resource
```

Command explanation

- `make:model Student` creates the Eloquent model class `Student`; `-m` also creates a migration for its table.
- `make:controller StudentController --resource` creates a controller with the methods index, create, store, show, edit, update and destroy.

The output confirms three files: the model `app/Models/Student.php`, a migration `database/migrations/2026_10_20_070330_create_students_table.php` and the controller `app/Http/Controllers/StudentController.php`.

```
ubuntu@ip-172-31-33-121:~/laravel-project$ php artisan make:model Student -m
php artisan make:controller StudentController --resource

  INFO  Model [app/Models/Student.php] created successfully.

  INFO  Migration [database/migrations/2026_10_20_070330_create_students_table.p
hp] created successfully.


  INFO  Controller [app/Http/Controllers/StudentController.php] created successf
ully.

ubuntu@ip-172-31-33-121:~/laravel-project$
```

Figure 26: *Generating the Student model, its migration and the resource controller.*

5.2 Defining the Students Table, Model and Controller

I then edited the generated files in `nano`. In the migration I defined the columns of the `students` table: an auto-increment `id`, a unique `student_id`, `name`, a unique `email`, `department`, a numeric `semester`, and `timestamps`.

```
    database/migrations/2026_10_20_070330_create_students_table.php *
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('student_id')->unique();
        $table->string('name');
        $table->string('email')->unique();
        $table->string('department');
        $table->unsignedTinyInteger('semester');
        $table->timestamps();
```

Figure 27: *Migration file defining the columns of the students table.*

In the `Student` model I added a `$fillable` list (`student_id`, `name`, `email`, `department`, `semester`). This tells Laravel which fields may be saved from a form in one step.



Figure 28: *The Student model with its fillable fields.*

In `StudentController` I wrote the CRUD logic. The part visible in the screenshot shows `index` returning the `students.index` view, `create` returning the `students.create` view, and `store` validating the form: `student_id` is required, up to 50 characters and unique; `name` and `department` are required strings; `email` must be a valid unique address; and `semester` must be an integer from 1 to 8.



Figure 29: *StudentController with the create and store methods and their validation rules.*

5.3 Migrating the Students Table

Now that the migration file described the table, I ran the migrations again so that the `students` table was created in MySQL.

Command used

```
php artisan migrate
php artisan migrate:status
```

Command explanation

- `migrate` runs only the migrations that have not yet been applied, so here it creates just the new table.
- `migrate:status` confirms which migrations have run and in which batch.

The first screenshot shows `2026_10_20_070330_create_students_table` marked DONE. The second shows the three default migrations in batch [1] and the new students migration in batch [2], all with the status **Ran**.

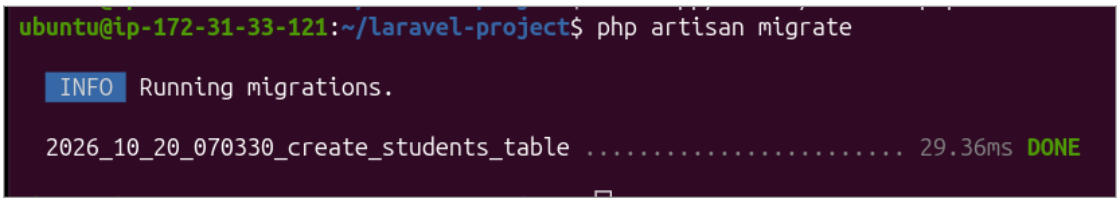


Figure 30: *Running the migration that creates the students table.*

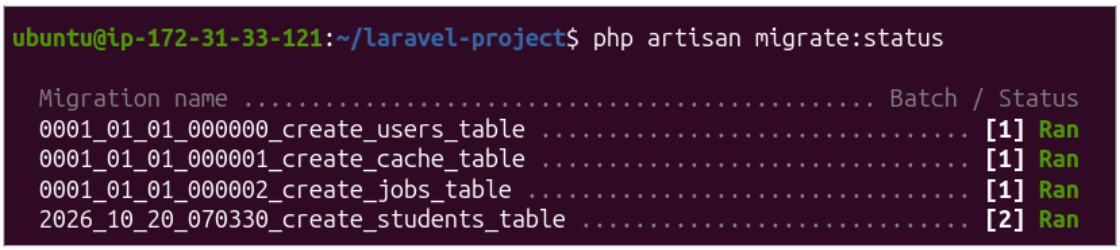


Figure 31: *Migration status showing the students table migration in batch 2.*

5.4 Defining the Routes

Routes connect browser URLs to controller methods. In `routes/web.php` I redirected the home page `/` to the students list and registered a resource route for the students controller.

The screenshot shows `Route::get('/', ...)` returning `redirect()->route('students.index')`, and `Route::resource('students', StudentController::class)`, which creates all CRUD routes with one line.

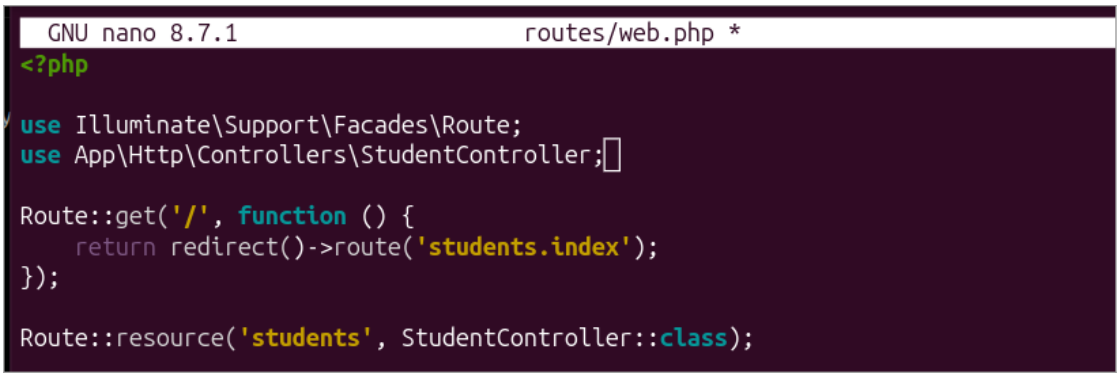


Figure 32: *The web.php file redirecting / to the students list and registering the students resource route.*

To confirm that the resource route generated what I expected, I listed the routes that start with `students`.

Command used

```
php artisan route:list --path=students
```

Command explanation

- `route:list` prints all registered routes.
- `--path=students` filters the list to routes whose URL contains `students`.

The output shows seven routes: `students.index`, `students.store`, `students.create`, `students.show`, `students.update` (PUT|PATCH), `students.destroy` (DELETE) and `students.edit`, each mapped to a `StudentController` method. The footer reads "Showing [7] routes".



Figure 33: *Listing the seven student routes with php artisan route:list.*

5.5 Creating the Blade Views

The user interface is built from Blade templates in `resources/views`. I first created a shared layout, `layout.blade.php`, which contains the page structure and CSS (a dark-blue header and a light page background) and the title "Student Management System". The other pages extend this layout.

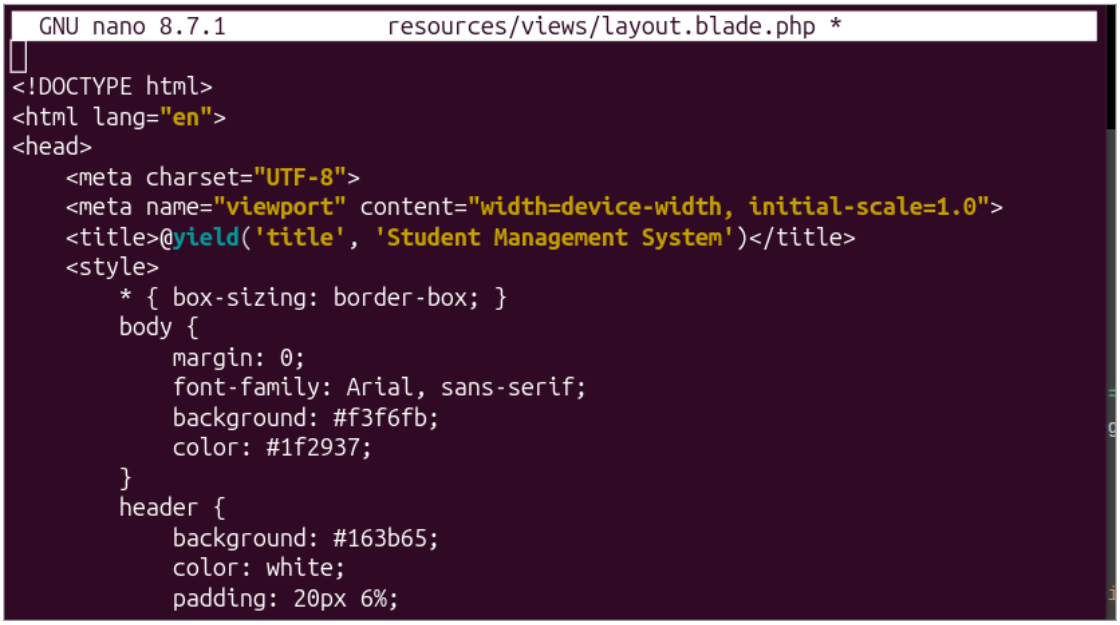


Figure 34: *The shared layout.blade.php template with the page styles.*

The students list page, `students/index.blade.php`, extends the layout, shows the heading "Students" with a "+ Add Student" button, and displays a message when there are no students yet.

```
  GNU nano 8.7.1        resources/views/students/index.blade.php *

@extends('layout')

@section('title', 'Students | Student Management System')

@section('content')
    <div class="topbar">
        <div>
            <h2>Students</h2>
            <p class="muted">View and manage registered students.</p>
        </div>
        <a href="{{ route('students.create') }}" class="btn">+ Add Student</a>
    </div>

    <div class="card">
        @if($students->isEmpty())
            <p>No students have been added yet. Select "Add Student" to create y>
        @else
            <div class="table-wrap">
```

Figure 35: *The students index Blade view.*

The "Add Student" page, `students/create.blade.php`, contains a POST form that submits to `students.store`. It includes the `@csrf` token and fields such as Student ID (maximum 50 characters).

```
  GNU nano 8.7.1       resources/views/students/create.blade.php *

@extends('layout')

@section('title', 'Add Student | Student Management System')

@section('content')
    <div class="card">
        <h2>Add Student</h2>
        <p class="muted">Enter the student's information below.</p>

        <form method="POST" action="{{ route('students.store') }}">
            @csrf

            <div class="form-group">
                <label for="student_id">Student ID</label>
                <input id="student_id" name="student_id" value="{{ old('student_>
                       required maxlength="50" placeholder="e.g. FA23-BSE-001">
            </div>
```

Figure 36: *The create Blade view containing the add-student form.*

Because Laravel compiles Blade templates into cached PHP files, I cleared the compiled views so that the new templates would be used.

Command used

```
php artisan view:clear
```

Command explanation

- `view:clear` deletes the compiled Blade views stored in the cache.

Laravel answered “Compiled views cleared successfully.”

```
ubuntu@ip-172-31-33-121:~/laravel-project$ php artisan view:clear

   INFO  Compiled views cleared successfully.

ubuntu@ip-172-31-33-121:~/laravel-project$
```

Figure 37: *Clearing the compiled Laravel views.*

I then created the details page, `students/show.blade.php`, which shows “Student Details” with a “Back to Students” button and a table of the record.

```
 GNU nano 8.7.1        resources/views/students/show.blade.php *

@extends('layout')

@section('title', 'Student Details | Student Management System')

@section('content')
    <div class="card">
        <div class="topbar">
            <div>
                <h2>Student Details</h2>
                <p class="muted">View the complete student record.</p>
            </div>
            <a class="btn btn-secondary" href="{{ route('students.index') }}">Ba>
        </div>

        <table>
```

Figure 38: *The show Blade view for a single student record.*

Finally I created the edit page, `students/edit.blade.php`. It is a POST form that uses `@method('PUT')` to submit to `students.update`, and its fields are pre-filled with the student’s current values.

```
 GNU nano 8.7.1        resources/views/students/edit.blade.php *

@extends('layout')

@section('title', 'Edit Student | Student Management System')

@section('content')
    <div class="card">
        <h2>Edit Student</h2>
        <p class="muted">Update the information for {{ $student->name }}.</p>

        <form method="POST" action="{{ route('students.update', $student) }}">
            @csrf
            @method('PUT')

            <div class="form-group">
                <label for="student_id">Student ID</label>
                <input id="student_id" name="student_id"
                       value="{{ old('student_id', $student->student_id) }}"
                       required maxlength="50">
```

Figure 39: *The edit Blade view with a PUT form for updating a student.*

5.6 Generating the Application Key

Laravel needs an application key to encrypt sessions and cookies, so I generated one.

Command used

```
php artisan key:generate
```

Command explanation

- `key:generate` creates a random key and stores it as `APP_KEY` in the `.env` file.

The message "Application key set successfully." confirms that the key was written.

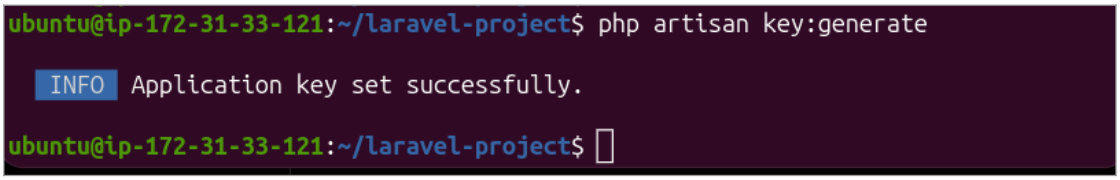


Figure 40: *Generating the Laravel application key.*

6. Testing and Running the Application

6.1 Running the Automated Tests

Before connecting Jenkins, I ran the project's automated tests by hand. Running them locally first shows that Jenkins will later run something that already works.

Command used

```
php artisan test
```

Command explanation

- `php artisan test` runs all PHPUnit tests in the `tests` folder and prints a summary.

The output shows `Tests\Unit\ExampleTest` ("that true is true") and `Tests\Feature\ExampleTest` ("homepage redirects to students" and "students page loads successfully") as PASS. The summary reads **3 passed (4 assertions)** with a duration of 0.19 s.

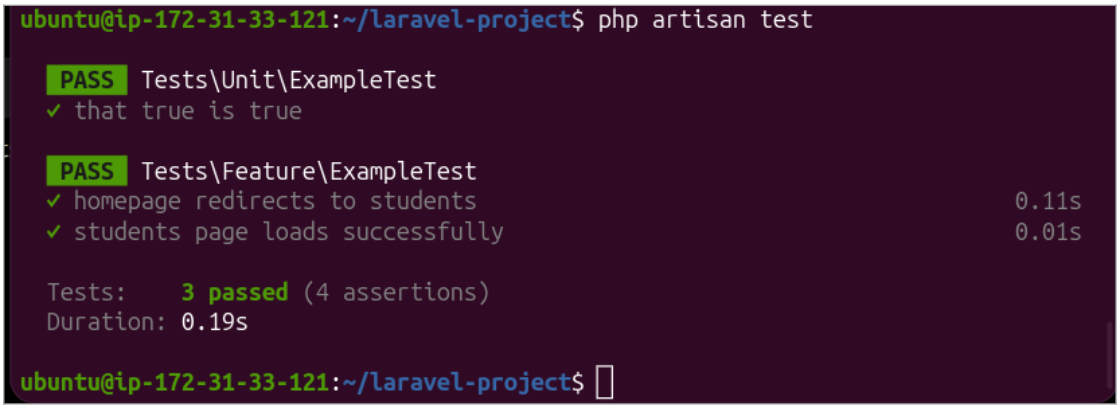


Figure 41: *All three Laravel tests passing with php artisan test.*

6.2 Clearing Caches and Re-checking Migrations

I changed to the project directory and cleared all cached bootstrap files so that Laravel would pick up the latest configuration, routes and views, then checked the migration status once more.

Command used

```
cd /home/ubuntu/laravel-project
php artisan optimize:clear
php artisan migrate:status
```

Command explanation

- `optimize:clear` removes the cached config, cache, compiled files, events, routes and views.
- `migrate:status` lists all migrations and their status.

Each cache type (config, cache, compiled, events, routes and views) is marked DONE, and all four migrations are listed as **Ran**.

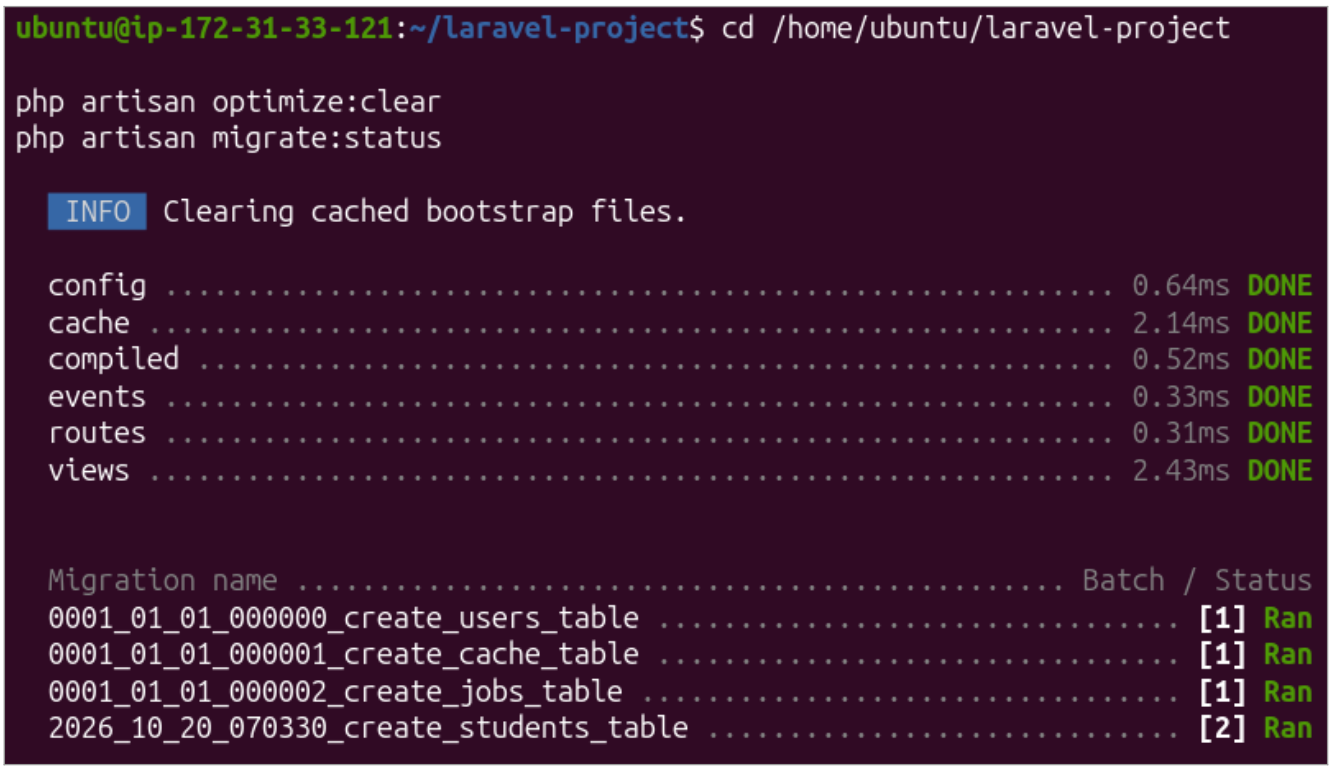


Figure 42: *Clearing Laravel caches and confirming that all migrations have run.*

6.3 Using the Application in the Browser

With the code and database ready, I opened the application in the browser on port 8001 at `http://13.51.206.252:8001/students`. I tested each part of the CRUD functionality. In the first screenshot the students page is shown with no records yet; the message “No students have been added yet” appears together with the “+ Add Student” button.

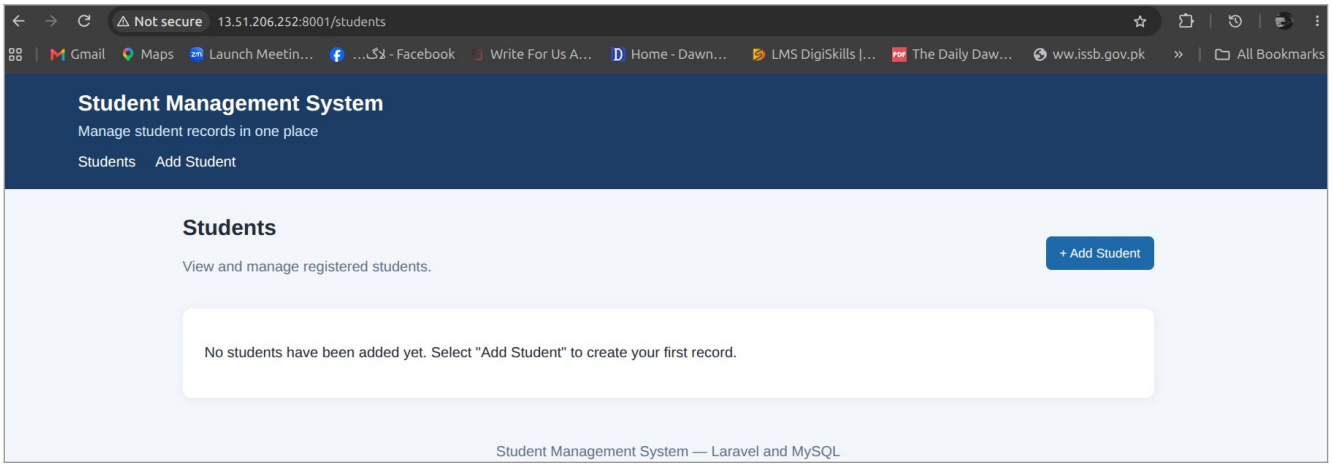


Figure 43: *The students page before any record has been added.*

Next I opened the "Add Student" form at `/students/create`. It contains the fields Student ID, Full Name, Email Address, Department and Semester, with "Save Student" and "Cancel" buttons.

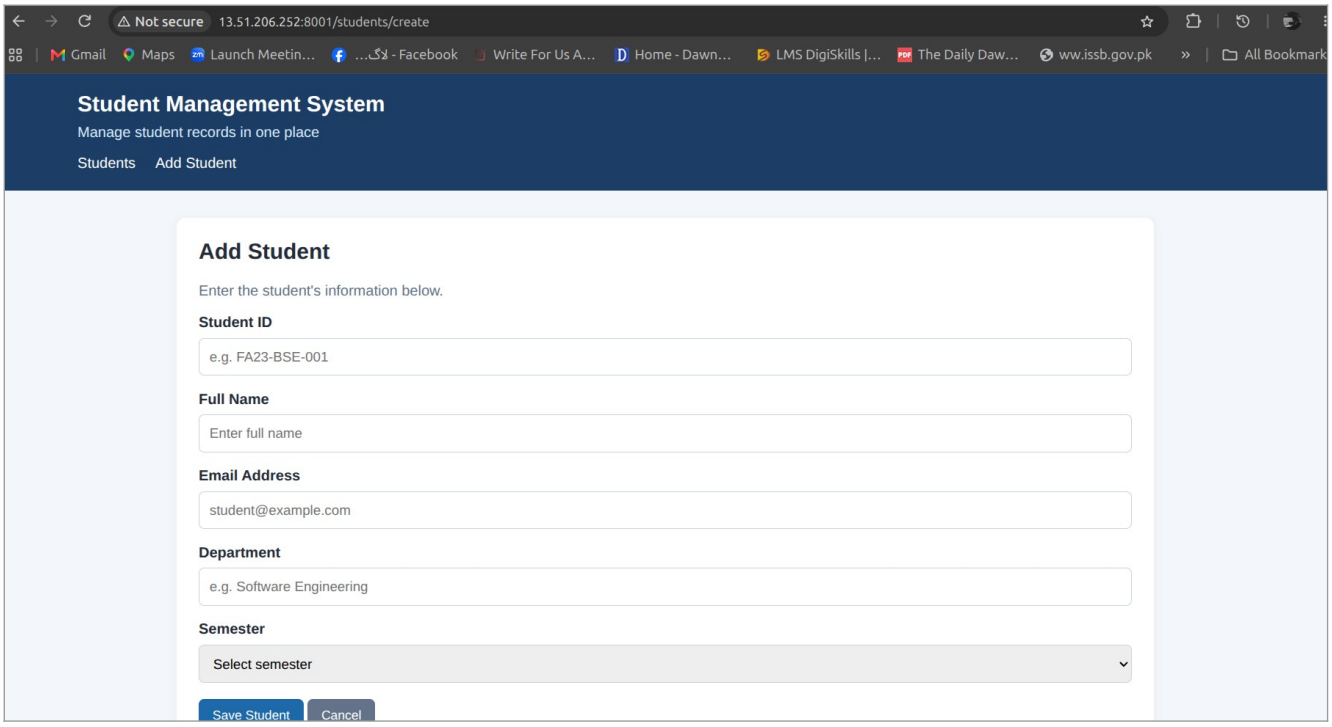


Figure 44: *The Add Student form served by the Laravel application.*

After saving a record, Laravel redirected to the list and displayed the green message "Student added successfully." The table shows the student `SP23-BSE-039` (Sami Khan, Computer Science, semester 8) with View, Edit and Delete buttons.

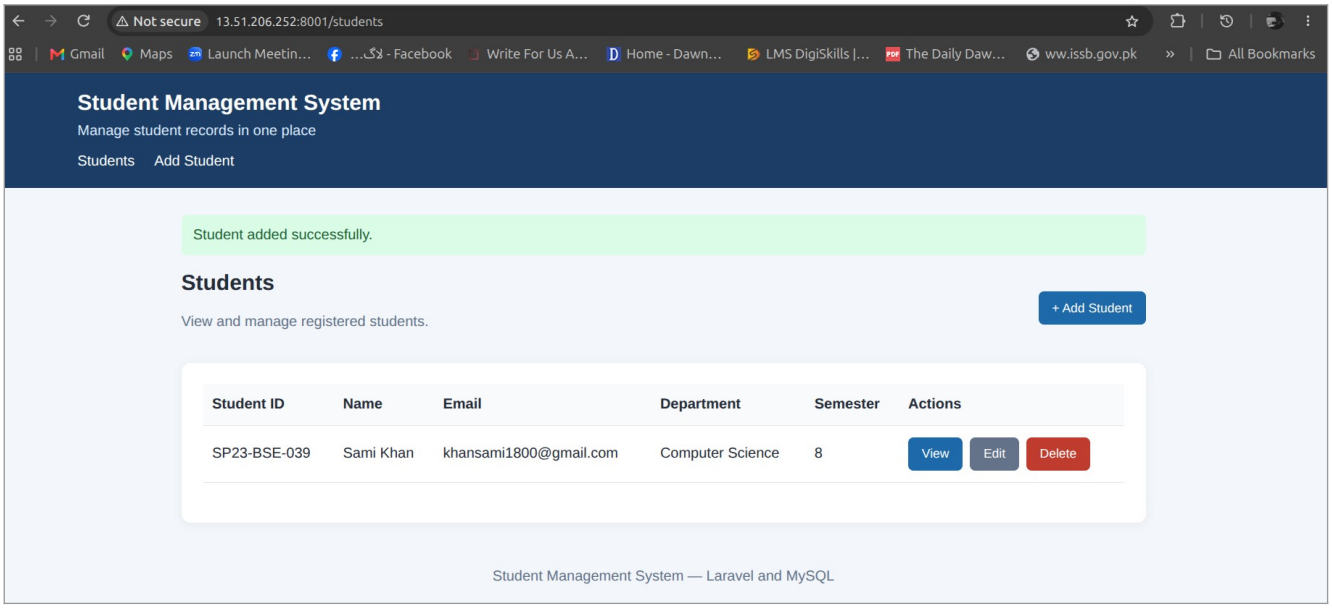


Figure 45: *The student list after the first record was added successfully.*

I also added a second student and opened its details with the View button (`/students/2`). The Student Details page shows the Student ID, Full Name, Email, Department, Semester and the created and last-updated timestamps (20 Oct 2026, 07:39 AM).

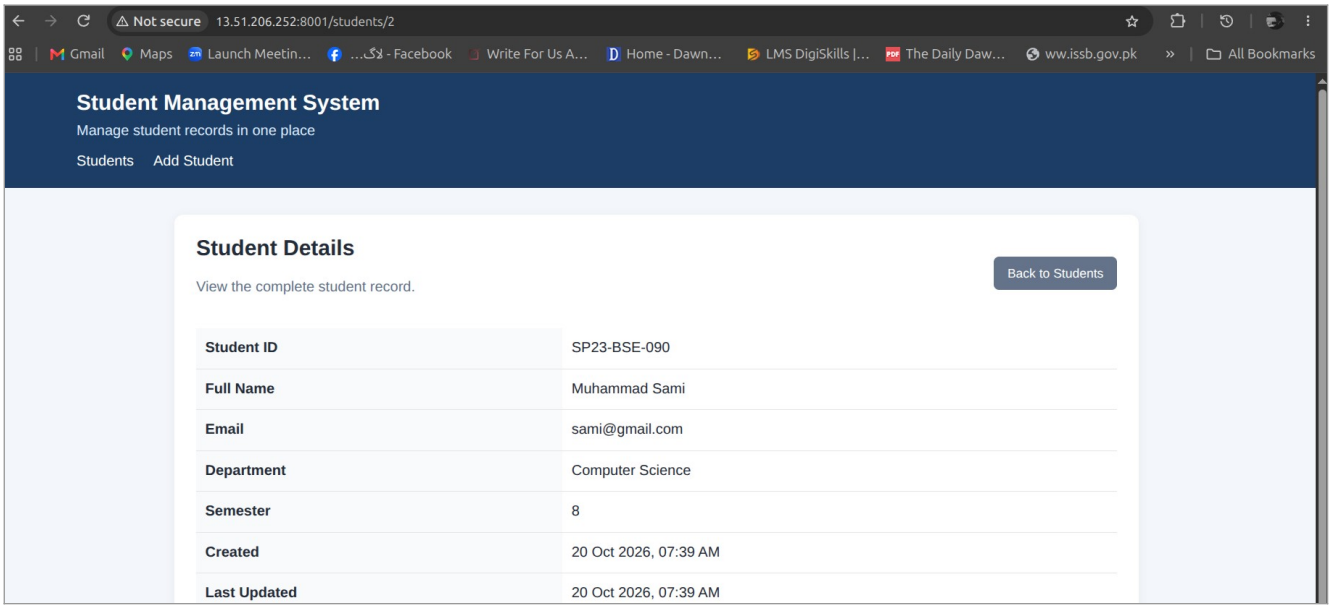


Figure 46: *Student Details page for the record with ID 2.*

Finally, I edited that record. The list now shows the second student (Muhammad Sami) with the ID `SP23-BSE-092`, whereas the details page above showed `SP23-BSE-090`, so the update was saved. Together with the earlier screenshots, this shows that create, read and update all work.

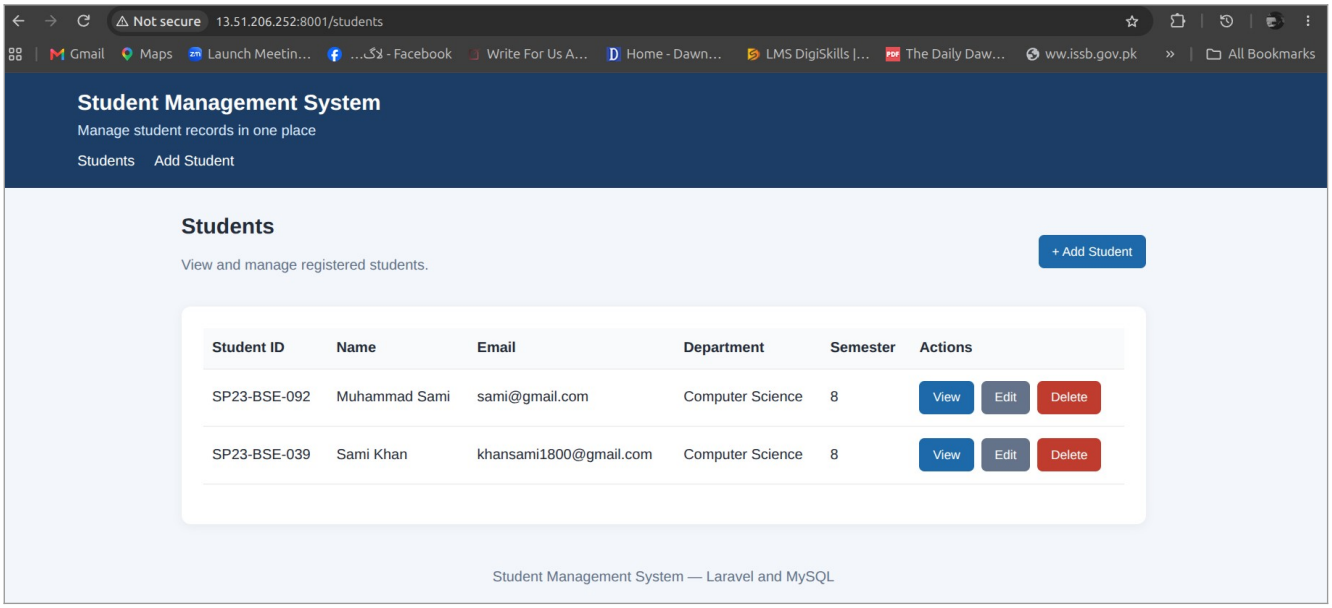


Figure 47: *The student list showing both records after the update.*

7. Pushing the Project to GitHub

Jenkins builds whatever is in the GitHub repository, so I pushed my work to the `main` branch. This step makes the code on GitHub identical to the code on the server.

Command used

```
git push origin main
```

Command explanation

- `git push` uploads local commits to a remote repository.
- `origin` is the name of the remote (my GitHub repository) and `main` is the branch being pushed.

Git asked for the GitHub username (`samikhan5055`) and password, which the terminal does not display. It then enumerated and compressed 38 objects, wrote 24 objects and printed `33c433b..62d30b7  main -> main`, which shows that the `main` branch on GitHub moved forward to commit `62d30b7`. (The commit commands that came before the push are not visible in this screenshot.)

```
ubuntu@ip-172-31-33-121:~/laravel-project$ git push origin main
Username for 'https://github.com': samikhan5055
Password for 'https://samikhan5055@github.com':
Enumerating objects: 38, done.
Counting objects: 100% (38/38), done.
Delta compression using up to 2 threads
Compressing objects: 100% (22/22), done.
Writing objects: 100% (24/24), 6.05 KiB | 6.05 MiB/s, done.
Total 24 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 5 local objects.
To https://github.com/samikhan5055/laravel-project.git
   33c433b..62d30b7  main -> main
```

Figure 48: *Pushing the local main branch to the GitHub repository.*

8. Creating the Jenkins Freestyle Jobs

I created three Freestyle jobs that form a simple chain: build, then test, then deploy. Each job is triggered automatically by the success of the previous one. This is the classic way to model a CI/CD flow before moving to a Pipeline.

8.1 JOB1-Laravel-Build

From the Jenkins dashboard I chose **New Item**, entered the name `JOB1-Laravel-Build` and selected the Freestyle project type. A Freestyle project runs a list of build steps that I configure in the web interface.

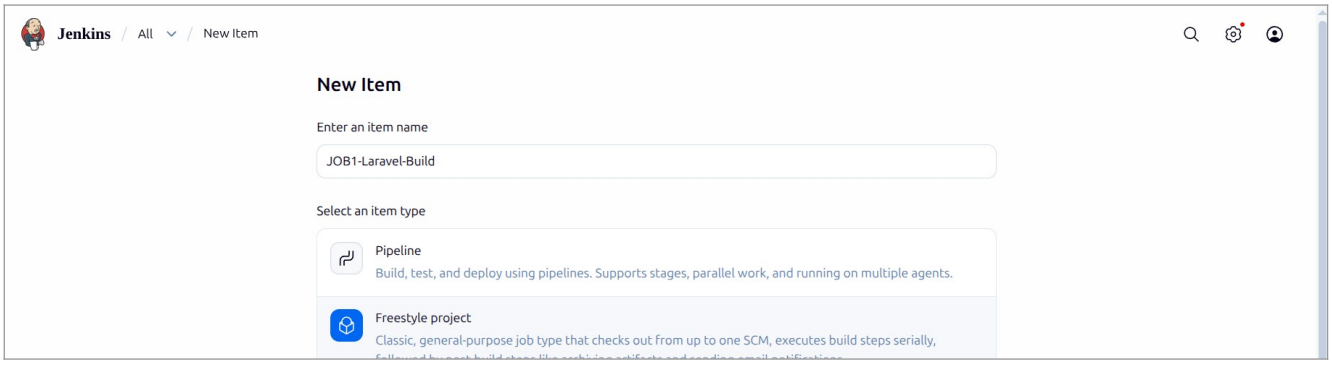


Figure 49: *Creating the new Freestyle item JOB1-Laravel-Build.*

In the Source Code Management section I selected Git and entered the repository URL `https://github.com/samikhan5055/laravel-project.git`. No credentials were needed because the repository is accessed over HTTPS, and the branch specifier is `*/main`, so the job always builds the main branch.

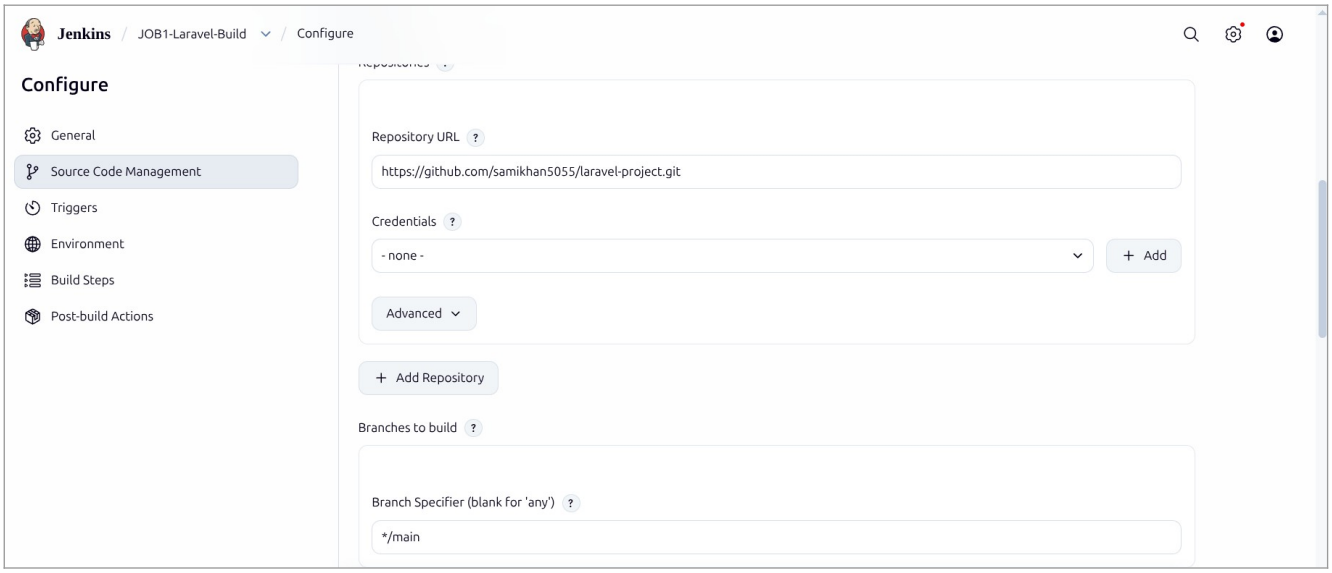


Figure 50: *JOB1 source code management settings pointing to the GitHub repository and the main branch.*

Under Build Steps I added an **Execute shell** step. The script begins with `#!/bin/bash` and `set -e` (stop at the first error) and prints "===== Checking PHP and Composer =====" before checking `php -v`. In this job the shell step checks the tools and installs the Composer dependencies of the project.

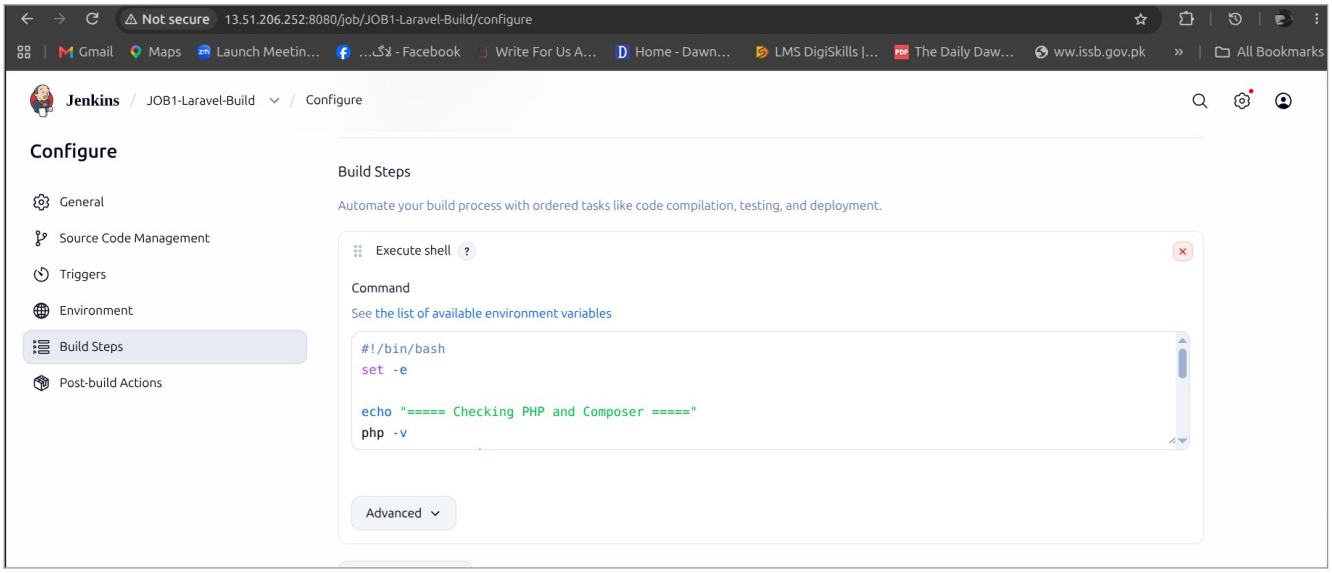


Figure 51: *JOB1 Execute shell build step.*

I started the job with a build. Build #1 completed with a green tick on 20 Oct 2026 at 07:52:24 and took 5.1 seconds. It checked out revision `62d30b7050ace6ff49794555f59a946e9510f588` from `refs/remotes/origin/main`, which is the commit I had just pushed.

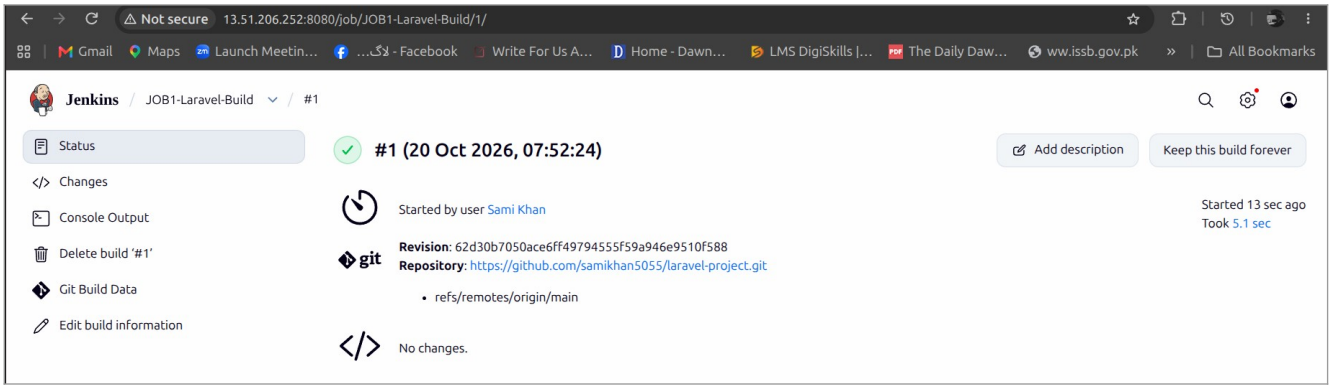


Figure 52: *JOB1 build #1 finished successfully on the pushed commit.*

The dashboard now lists `JOB1-Laravel-Build` with its last success (#1), no failure and a duration of 5.1 seconds.

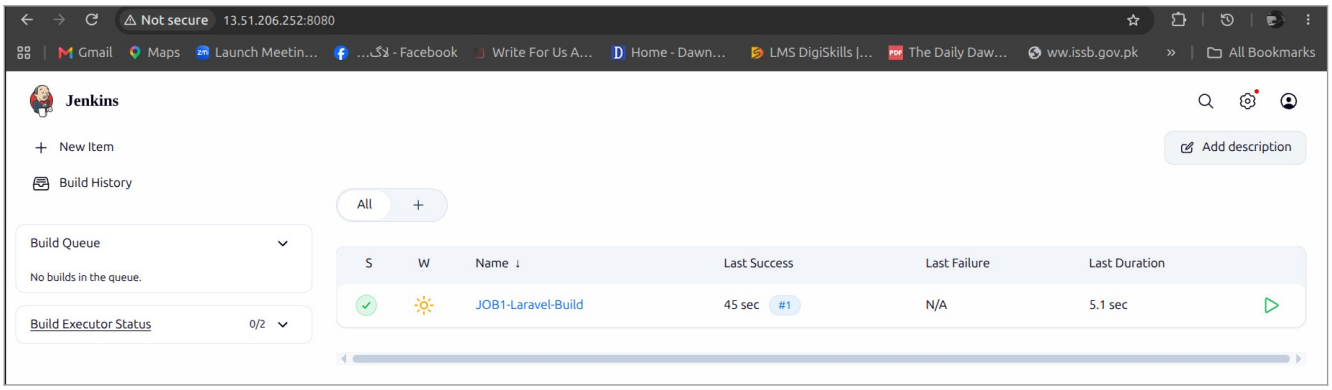


Figure 53: *Jenkins dashboard showing JOB1-Laravel-Build with a successful build.*

8.2 JOB2-Laravel-Test

The second job runs the automated tests. I created `JOB2-Laravel-Test` as another Freestyle project.

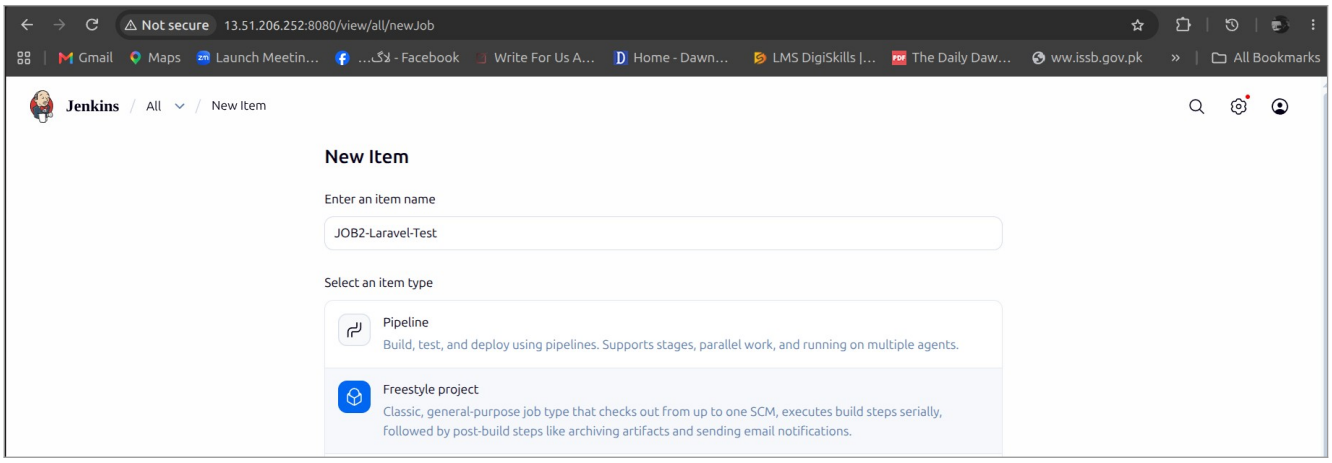


Figure 54: *Creating the Freestyle item JOB2-Laravel-Test.*

It uses the same GitHub repository as JOB1.

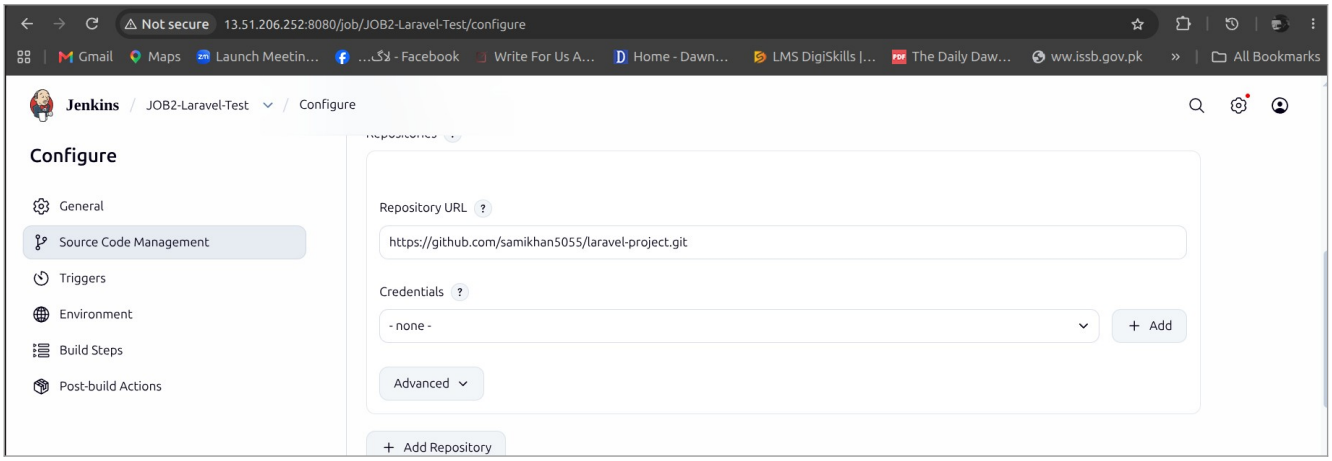


Figure 55: *JOB2 source code management settings using the same repository URL.*

In the Triggers section I enabled **Build after other projects are built**, entered `JOB1-Laravel-Build` as the project to watch, and chose “Trigger only if build is stable”. This chains the jobs: JOB2 starts automatically only when JOB1 succeeds.

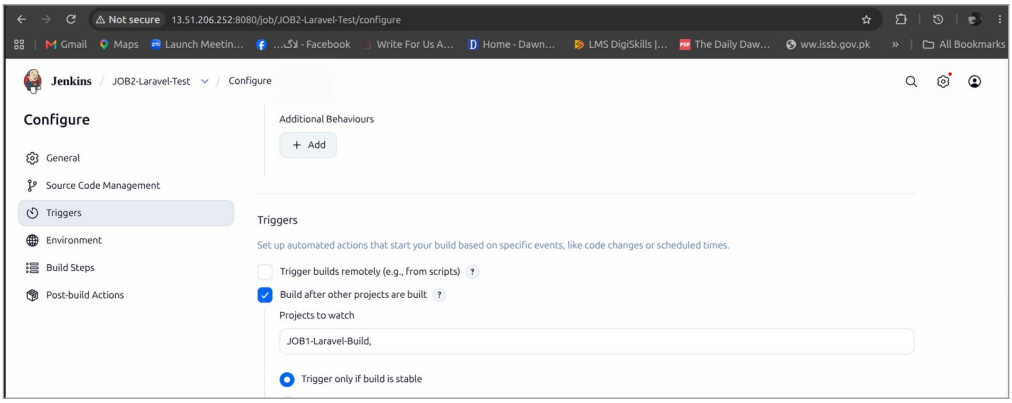


Figure 56: *JOB2 trigger configured to run after JOB1-Laravel-Build.*

Its Execute shell step starts with `set -e`, prints "===== Installing dependencies =====", runs `composer install --no-interaction --prefer-dist`, and then runs the Laravel tests.

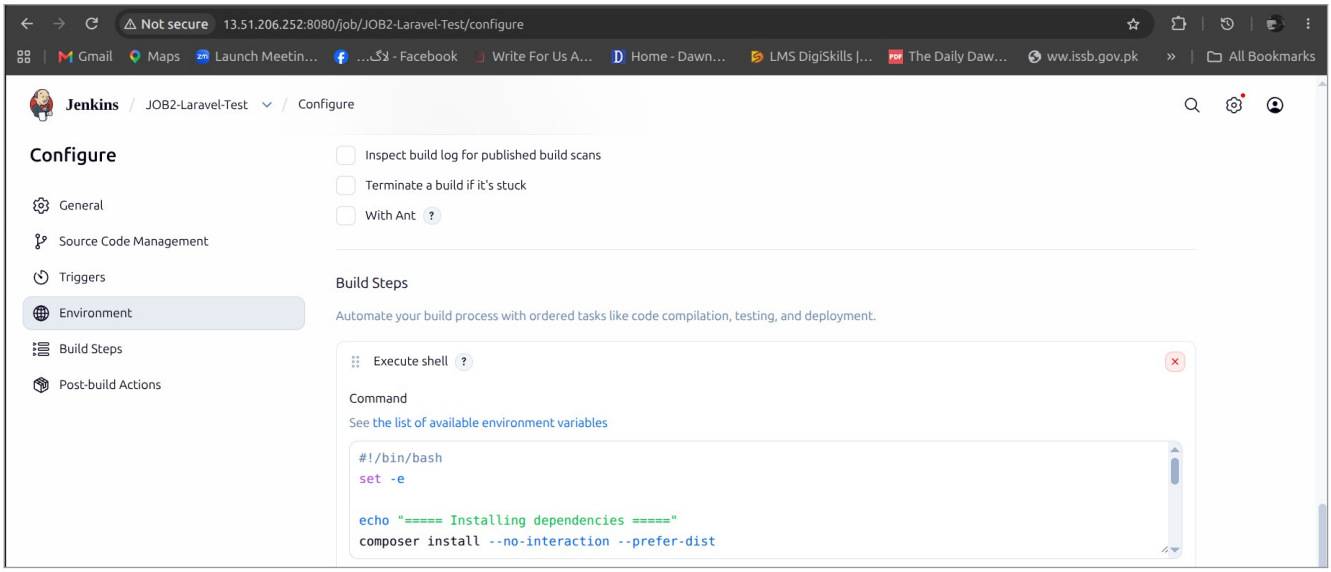


Figure 57: *JOB2 Execute shell build step installing dependencies.*

Build #1 of JOB2 succeeded on 20 Oct 2026 at 07:59:08 in 3.1 seconds, using the same revision `62d30b7`.

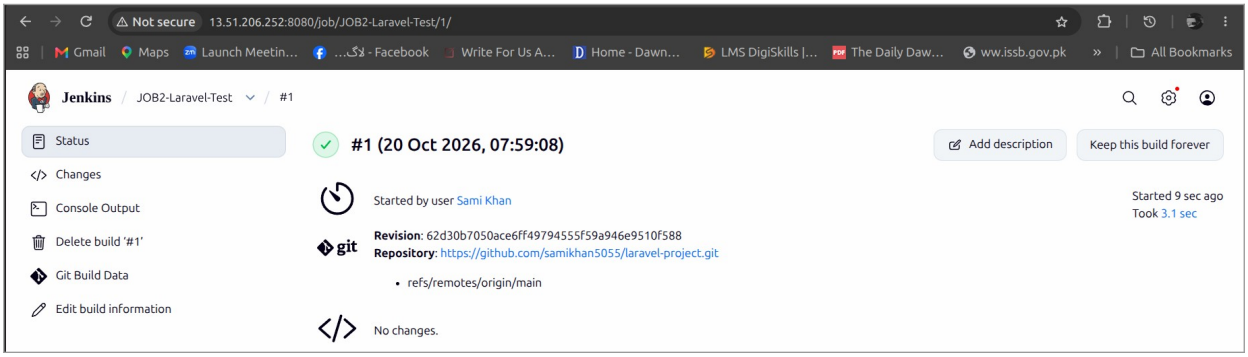


Figure 58: *JOB2 build #1 finished successfully.*

The console output shows "===== Running Laravel tests =====". Both `Tests\Unit\ExampleTest` and `Tests\Feature\ExampleTest` pass: "that true is true", "homepage redirects to students" and "students page loads successfully". The summary is **3 passed (4 assertions)**, followed by "===== All tests passed =====" and "Finished: SUCCESS". The odd characters such as `[32;1m` are the colour codes of the Laravel output, which the Jenkins console shows as plain text.

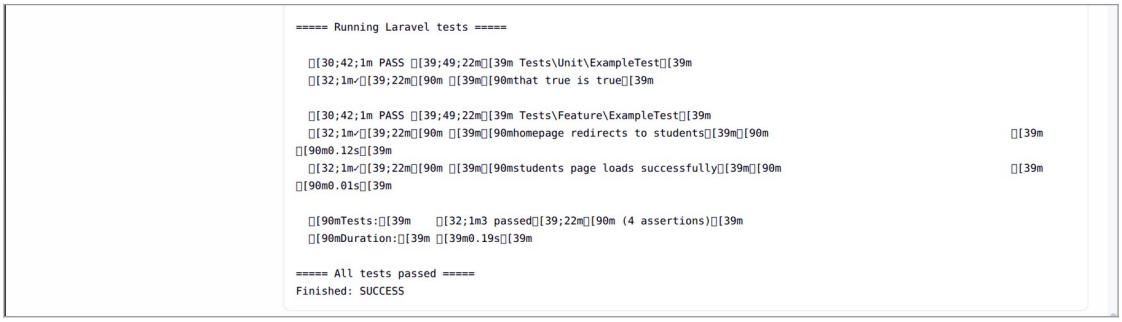


Figure 59: *Console output of JOB2 showing all three Laravel tests passing.*

The dashboard now shows both `JOB1-Laravel-Build` (#1) and `JOB2-Laravel-Test` (#1) as successful.

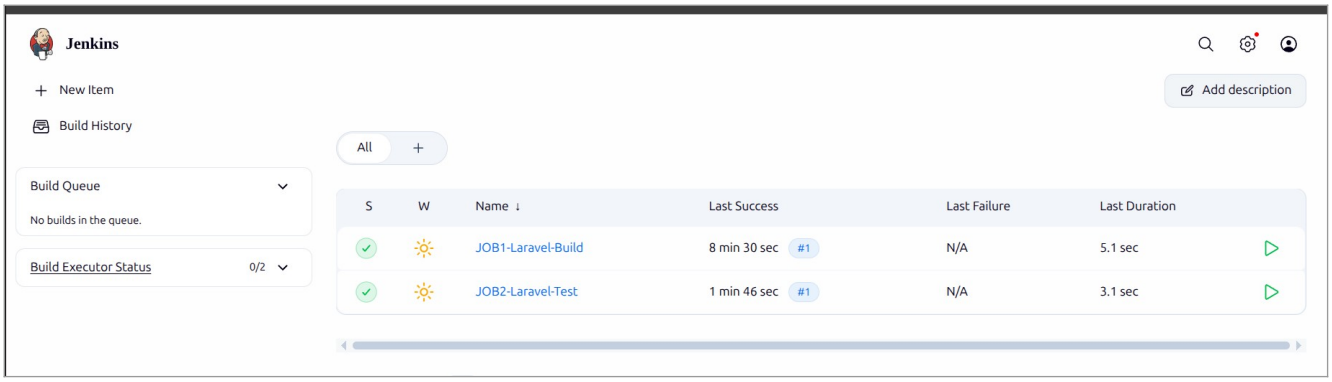


Figure 60: *Dashboard showing JOB1 and JOB2 with successful builds.*

8.3 JOB3-Laravel-Deploy

The third job is the deployment stage. I created `JOB3-Laravel-Deploy` as a Freestyle project.

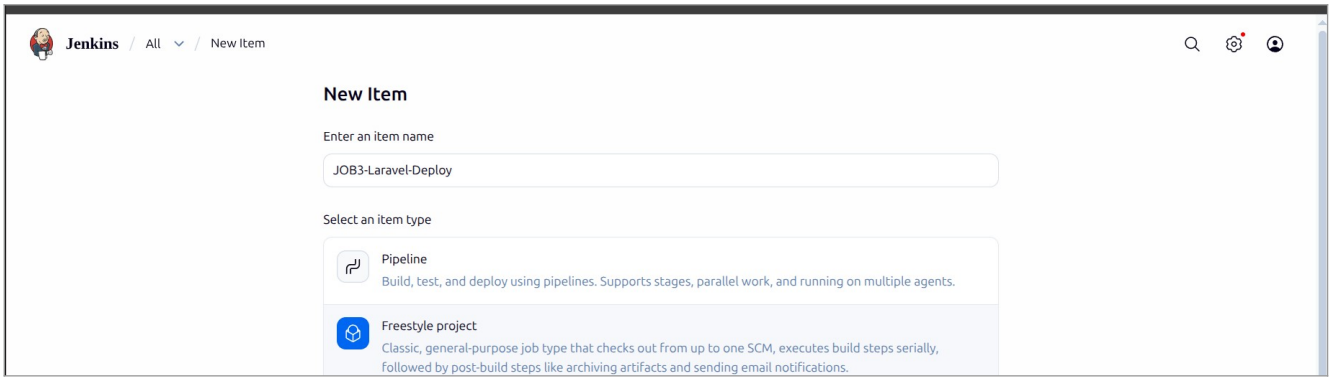


Figure 61: *Creating the Freestyle item JOB3-Laravel-Deploy.*

Like JOB2, it is chained to the previous job: it watches `JOB2-Laravel-Test` and is triggered only if that build is stable.

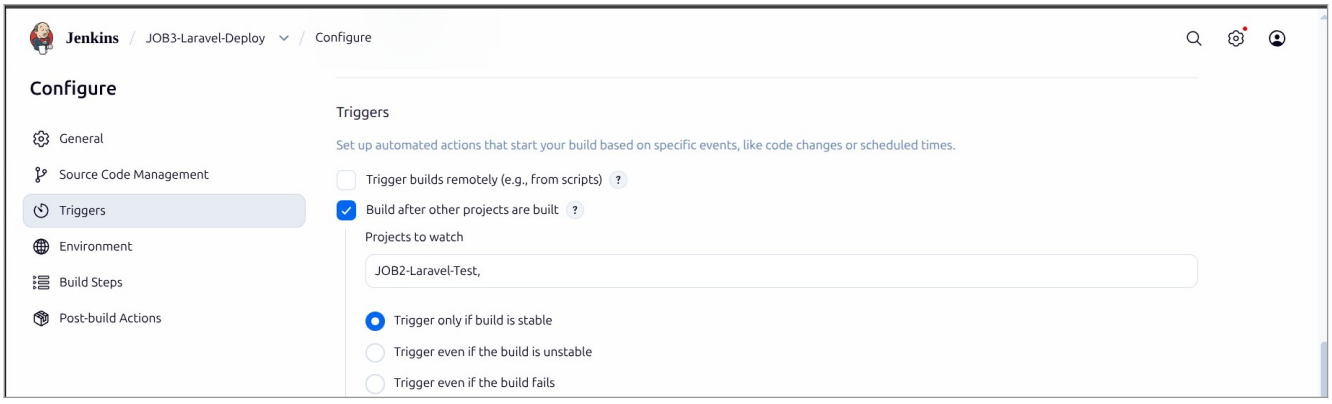


Figure 62: *JOB3 trigger configured to run after JOB2-Laravel-Test.*

Its Execute shell step starts with `set -e` and prints "`===== Verifying deployment workspace =====`" before running `pwd` to show the working directory.

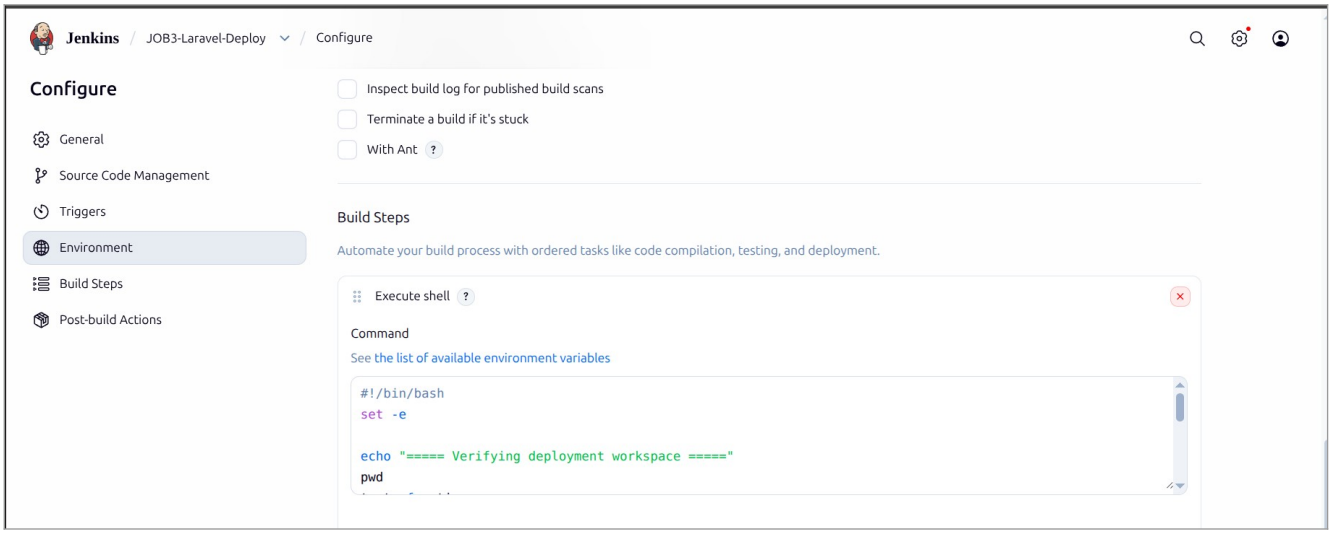


Figure 63: *JOB3 Execute shell build step verifying the deployment workspace.*

Build #2 of JOB3 completed successfully on 20 Oct 2026 at 08:06:15 and took 2.6 seconds.

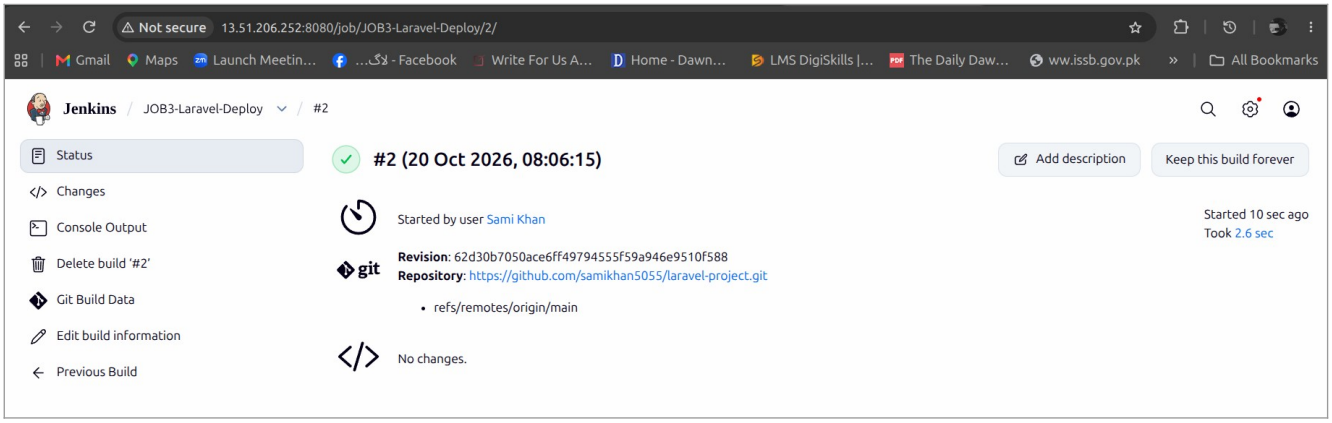


Figure 64: *JOB3 build #2 finished successfully.*

The console output shows the steps that this job performs: it installs the dependencies (ending with the Composer funding message), prints "===== Verifying Laravel application =====" with **Laravel Framework 13.37.0**, then checks the student routes and lists all seven of them. It ends with "===== Deployment prerequisites verified =====" and "Finished: SUCCESS". So this Freestyle job verifies that the application is ready for deployment; it does not itself copy files to the web directory.

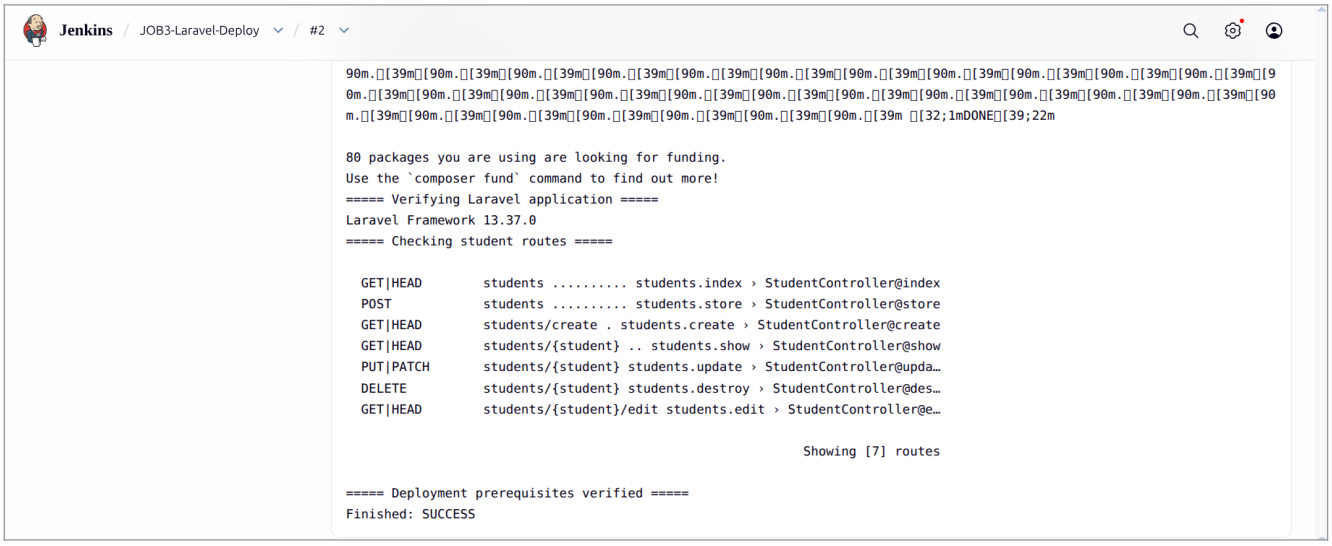


Figure 65: *Console output of JOB3 verifying Laravel and the student routes.*

The dashboard shows all three jobs with a successful last build. For `JOB3-Laravel-Deploy` the last success is #2, and the dashboard also lists build #1 as its last failure; build #2 is the successful run documented here.

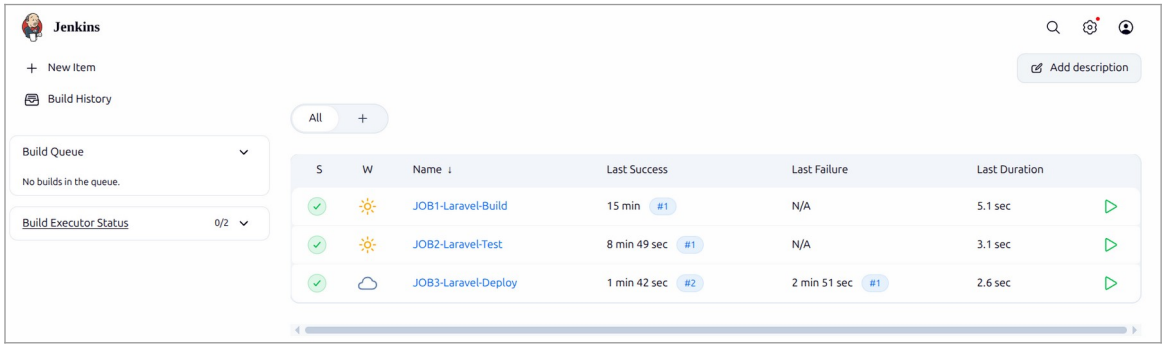


Figure 66: *Dashboard showing JOB1, JOB2 and JOB3 with successful builds.*

9. Creating the Jenkins Pipeline

9.1 Creating the Pipeline Job

Freestyle jobs are configured in the browser, whereas a **Pipeline** keeps the whole workflow in a text file, the `Jenkinsfile`, stored in the repository. This lets the CI/CD process be versioned together with the code. I created a new item named `Laravel-Student-Pipeline` and selected the **Pipeline** type.

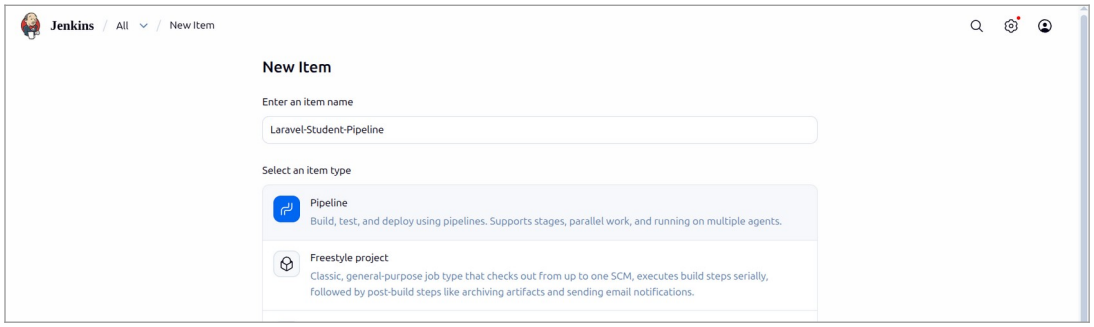


Figure 67: *Creating the Pipeline item Laravel-Student-Pipeline.*

In the Pipeline section I set the definition to “Pipeline script from SCM”, selected Git as the SCM and entered the repository URL `https://github.com/samikhan5055/laravel-project.git`. Jenkins therefore reads the pipeline definition (the `Jenkinsfile`) directly from the GitHub repository.

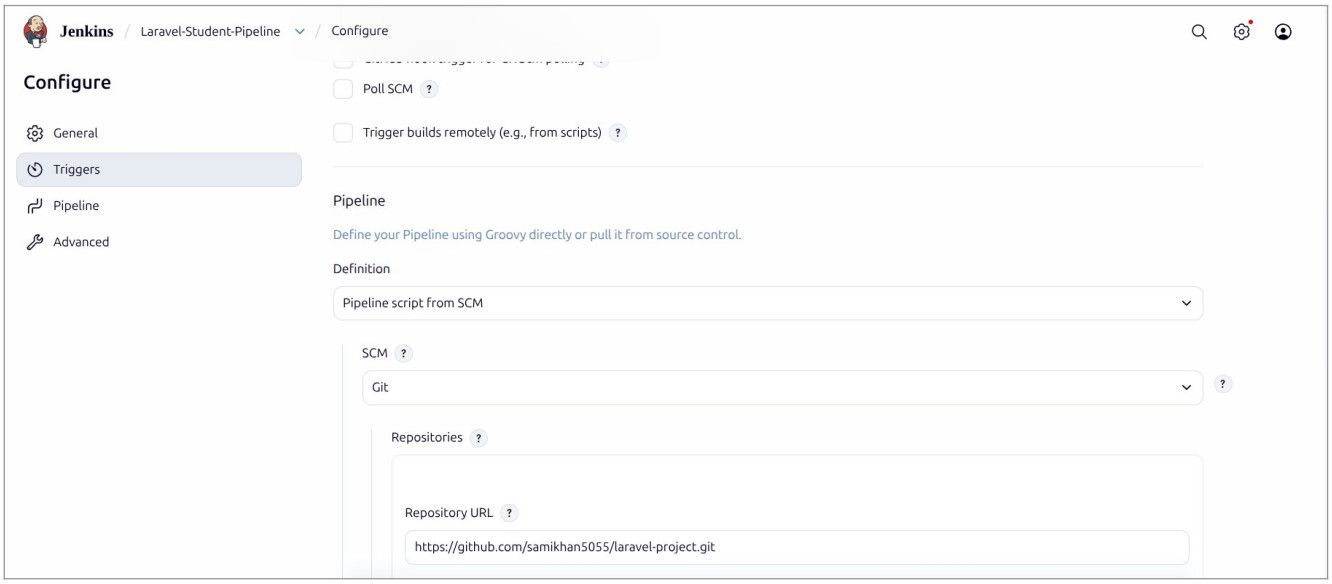


Figure 68: *Pipeline configured to load its script from the GitHub repository.*

9.2 Installing Nginx and PHP-FPM on the Server

The pipeline’s last stage deploys the application to a web server on the same EC2 instance. For that I installed **Nginx**, which serves web requests, and **PHP 8.5-FPM**, which executes the PHP code. The screenshot shows only the result of the `apt` installation (the command itself is not visible): apt lists `nginx` and `php8.5-fpm` under “Installing”, adds the dependency `nginx-common`, and reports 3 packages to install with 14.2 MB of extra space needed.

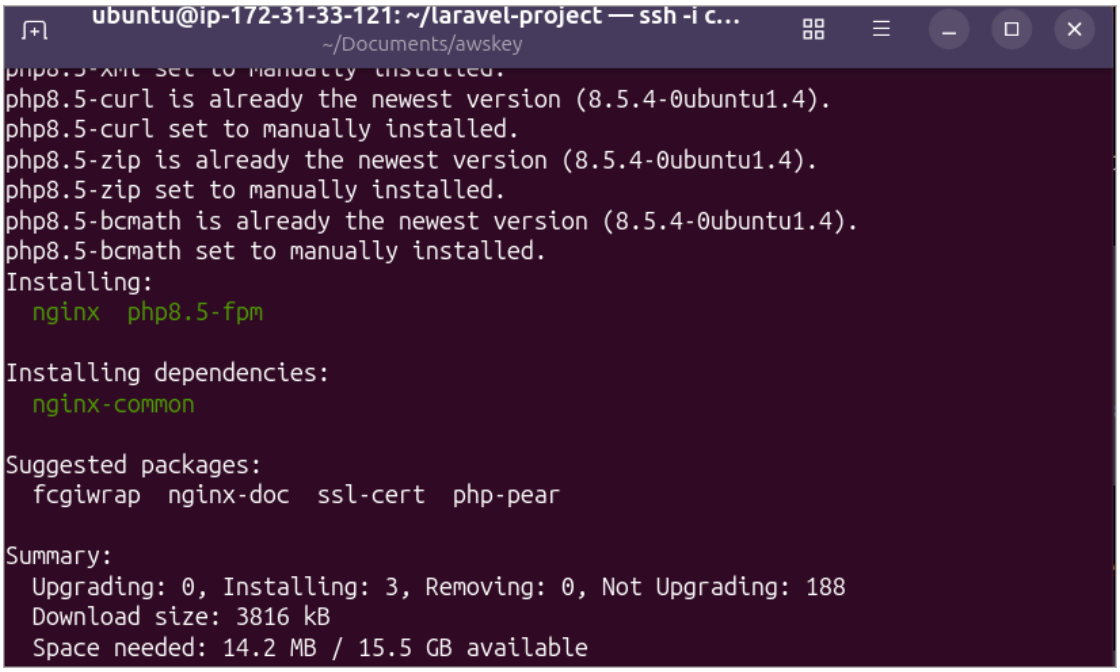


Figure 69: *Apt installing nginx and php8.5-fpm with the nginx-common dependency.*

I then started both services and made them start at boot, checked that they were running, and inspected the permissions of the Laravel writable folders.

Command used

```
sudo systemctl enable --now nginx
sudo systemctl enable --now php8.5-fpm
systemctl is-active nginx
systemctl is-active php8.5-fpm
cd /home/ubuntu/laravel-project
ls -ld storage bootstrap/cache
```

Command explanation

- `systemctl enable --now <service>` starts the service immediately (`--now`) and also enables it at boot.
- `systemctl is-active <service>` prints `active` when the service is running.
- `ls -ld storage bootstrap/cache` shows the owner and permissions of the two folders that Laravel must be able to write to.

Both `is-active` checks print `active`. The `ls -ld` output lists `bootstrap/cache` and `storage` as directories (`drwxrwxr-x`) owned by `ubuntu`.

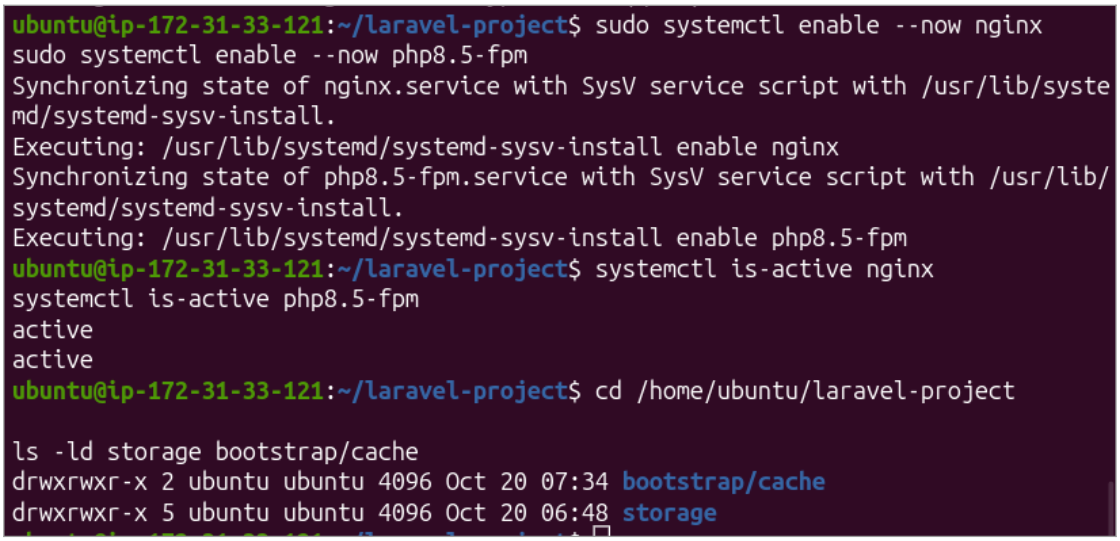


Figure 70: *Starting Nginx and PHP-FPM and checking the Laravel writable folders.*

9.3 Writing the Jenkinsfile

I then wrote the `Jenkinsfile` in `nano`. It defines the environment variable `DEPLOY_DIR = '/var/www/student-management'` (the production folder) and four stages: **Checkout**, **Build**, **Test** and **Deploy to EC2**. The file is shown in three screenshots.

The **Checkout** stage runs `checkout scm` to fetch the repository. The **Build** stage runs `composer install --no-interaction --prefer-dist --optimize-autoloader`, copies `.env.example` to `.env` if no `.env` exists, creates an empty `database/database.sqlite`, and exports testing settings (`DB_CONNECTION=sqlite`, `SESSION_DRIVER=file`, `CACHE_STORE=file`, `QUEUE_CONNECTION=sync`). It generates an application key only if none exists and clears the config cache. Using SQLite here means that the tests never touch the production MySQL database.

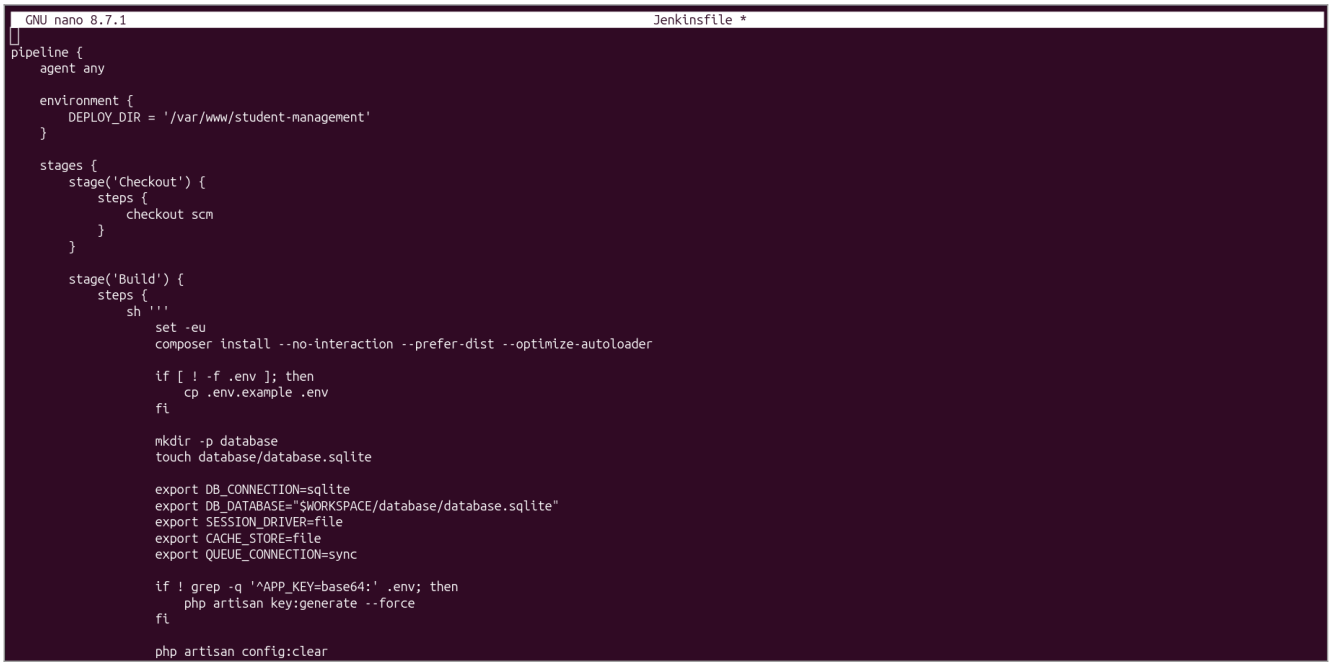


Figure 71: *First part of the Jenkinsfile: environment, Checkout stage and the start of the Build stage.*

The **Test** stage exports the same SQLite settings, then runs `php artisan migrate --force` and `php artisan test`. The **Deploy to EC2** stage begins by checking that the production `.env`, `storage` and `bootstrap/cache` exist in `$DEPLOY_DIR`, and then synchronises the files with `rsync`.

Key deployment command

```
rsync -a --delete \
    --exclude='/.env' \
    --exclude='/storage/' \
    --exclude='/bootstrap/cache/' \
    --exclude='/.git/' \
    --exclude='/database/database.sqlite' \
    "$WORKSPACE/" "$DEPLOY_DIR/"
```

Command explanation

- `rsync -a` copies files recursively while keeping permissions and timestamps.
- `--delete` removes files in the destination that no longer exist in the source, so the live folder matches the repository.
- Each `--exclude` protects something that must stay on the server: the private production `.env`, the `storage` and `bootstrap/cache` folders, the `.git` folder and the SQLite test database.
- `"$WORKSPACE/" "$DEPLOY_DIR/"` is the source (the Jenkins workspace) and the destination (`/var/www/student-management`).

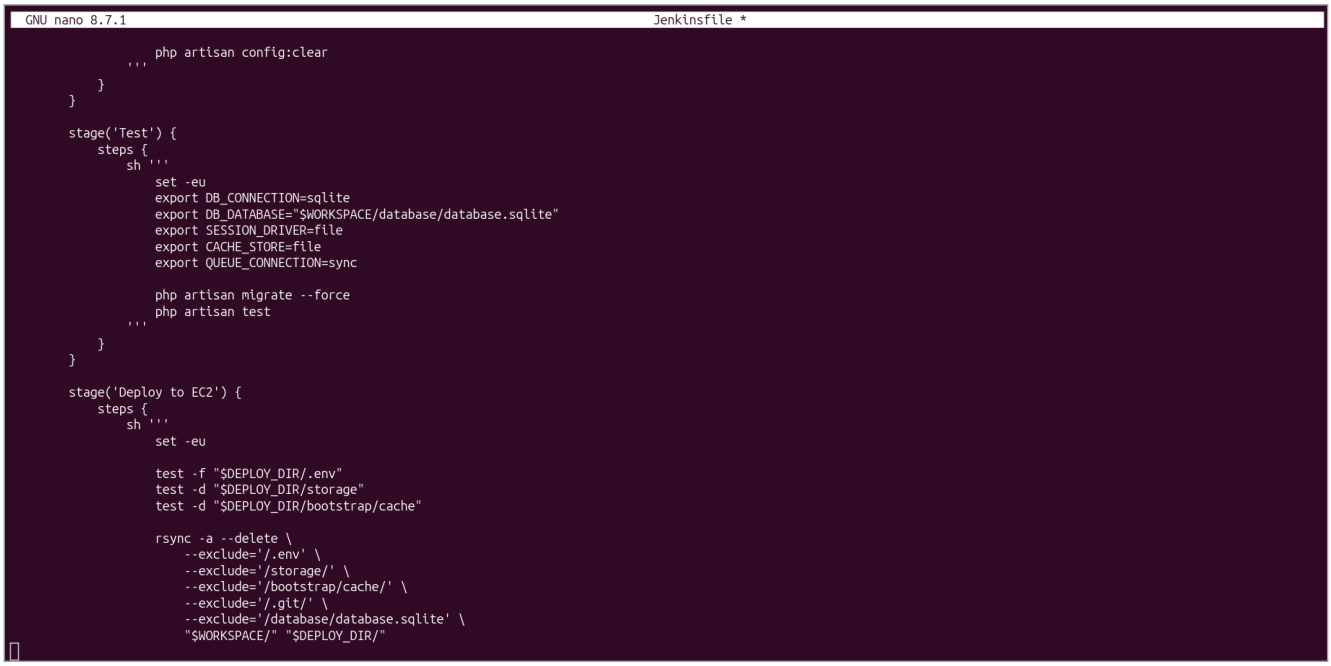


Figure 72: *Second part of the Jenkinsfile: Test stage and the start of the Deploy to EC2 stage.*

After the copy, the stage changes to `$DEPLOY_DIR` and runs `php artisan optimize:clear`, `config:cache`, `route:cache` and `view:cache` to refresh the Laravel caches, then prints “Deployment files updated successfully.” The `post` section prints a message on success, on failure, and always at the end of the run.

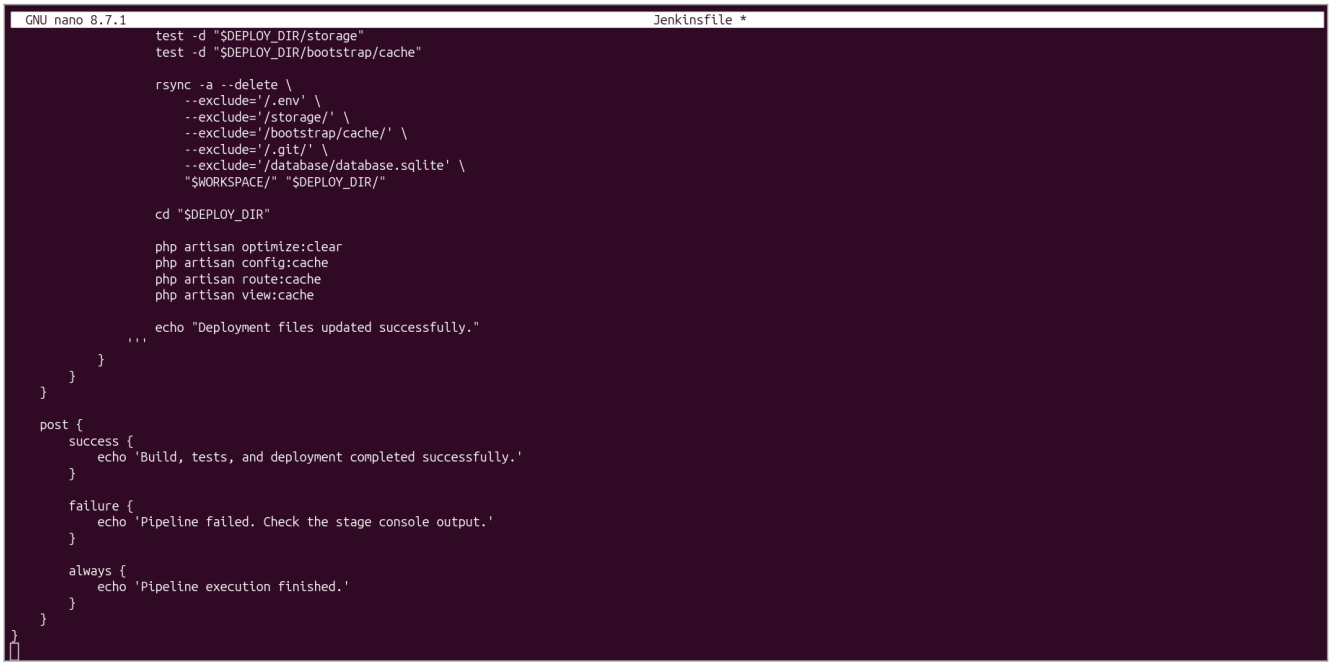


Figure 73: *Final part of the Jenkinsfile: cache commands and the post actions.*

9.4 Committing the Jenkinsfile and Running the Pipeline

Jenkins reads the `Jenkinsfile` from GitHub, so it had to be committed and pushed first.

Command used

```
cd /home/ubuntu/laravel-project
git add Jenkinsfile
git commit -m "Replace Jenkins pipeline with EC2 deployment"
git push origin main
```

Command explanation

- `git add Jenkinsfile` stages the file for the next commit.
- `git commit -m "..."` records the staged change with a message.
- `git push origin main` uploads the commit to the `main` branch on GitHub.

Git created commit `862d9fe` on `main` (1 file changed, 23 insertions, 108 deletions). The message about the committer name and email is only a warning that Git set them automatically from the server's username and hostname. The push result itself is not visible in this screenshot.

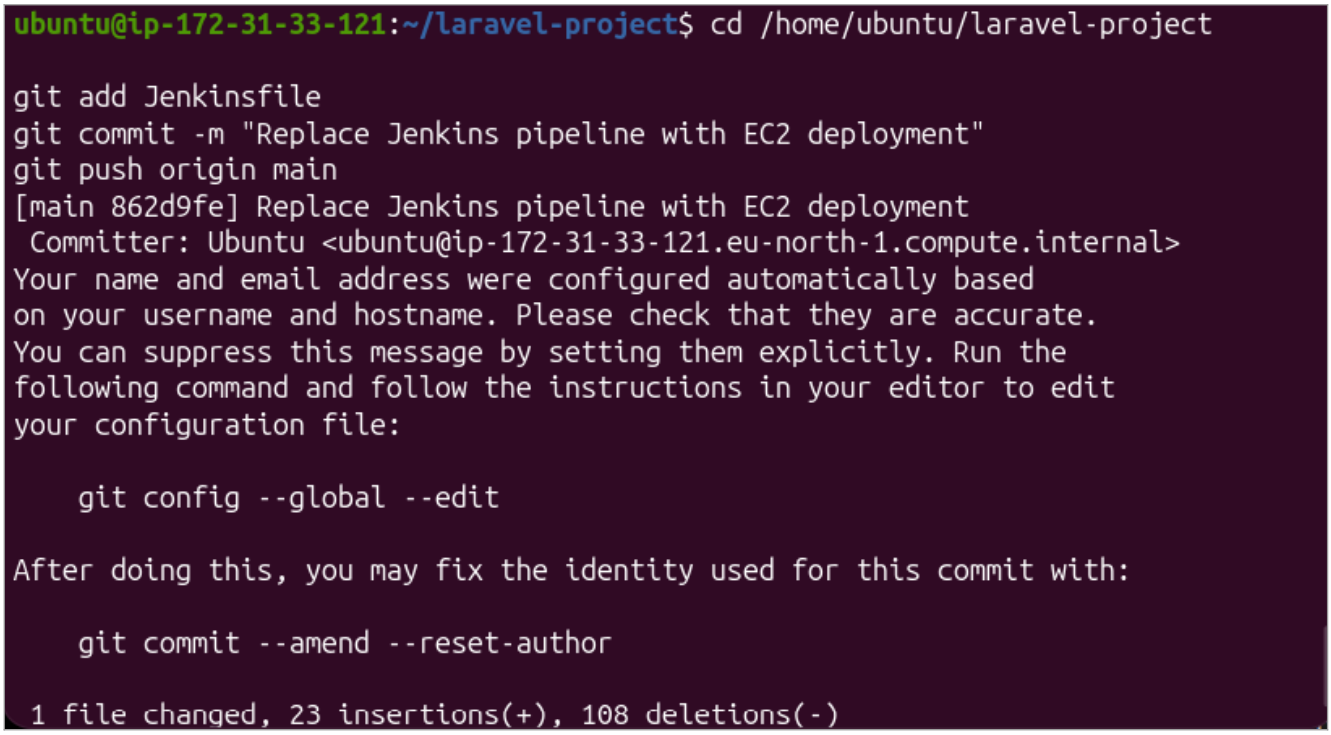


Figure 74: *Committing the Jenkinsfile and pushing it to GitHub.*

I then ran **Build Now** on `Laravel-Student-Pipeline`. The pipeline page shows that its first two builds, #1 (08:32) and #2 (08:34), failed, while build #3 (08:40) succeeded. The permalinks list build #3 as the last stable and last successful build.

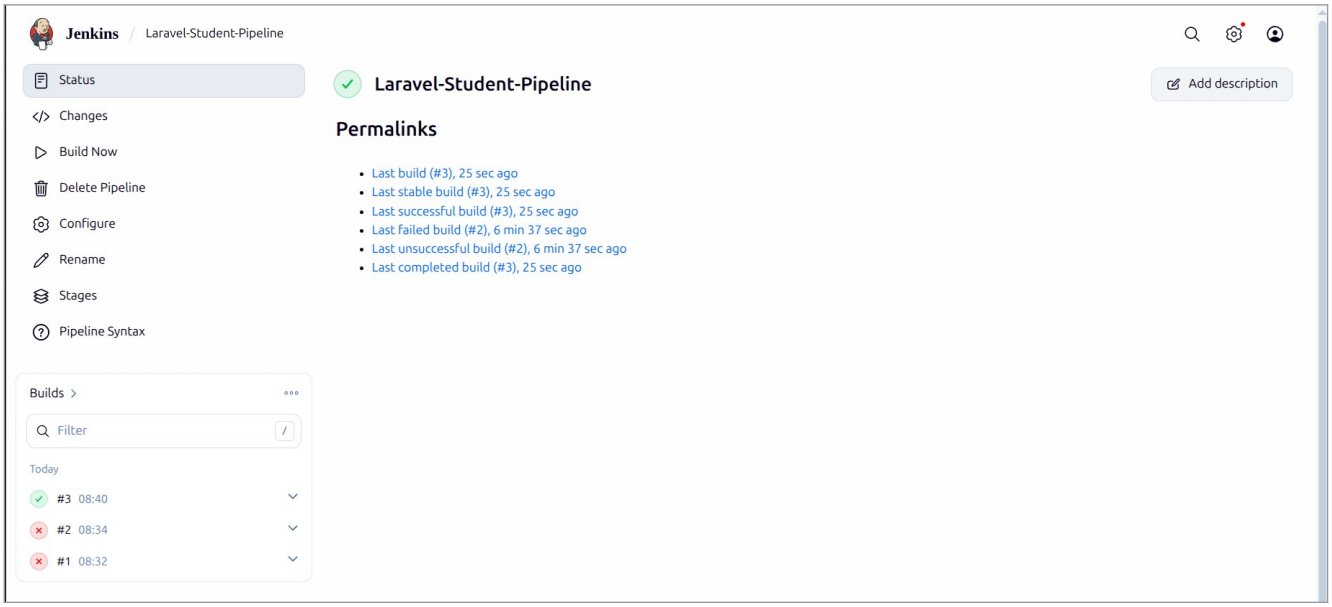


Figure 75: *Pipeline status showing builds #1 and #2 as failed and build #3 as successful.*

Build #3 started at 08:40:39 and took 9.1 seconds. It used revision `d2738c2a689f1dca6a7d6c82b5176aa92da5854c` from `refs/remotes/origin/main`, and the commit shown is "Update Jenkins pipeline deployment", so the `Jenkinsfile` was updated once more before this successful run.



Figure 76: *Pipeline build #3 details with its commit and revision.*

The end of the console output of build #3 shows "Showing [7] routes", then "Deployment completed successfully.", the post-action messages "Pipeline execution finished." and "SUCCESS: Build, tests, and EC2 deployment completed.", and finally "Finished: SUCCESS". This confirms that all stages, including the deployment stage, completed.

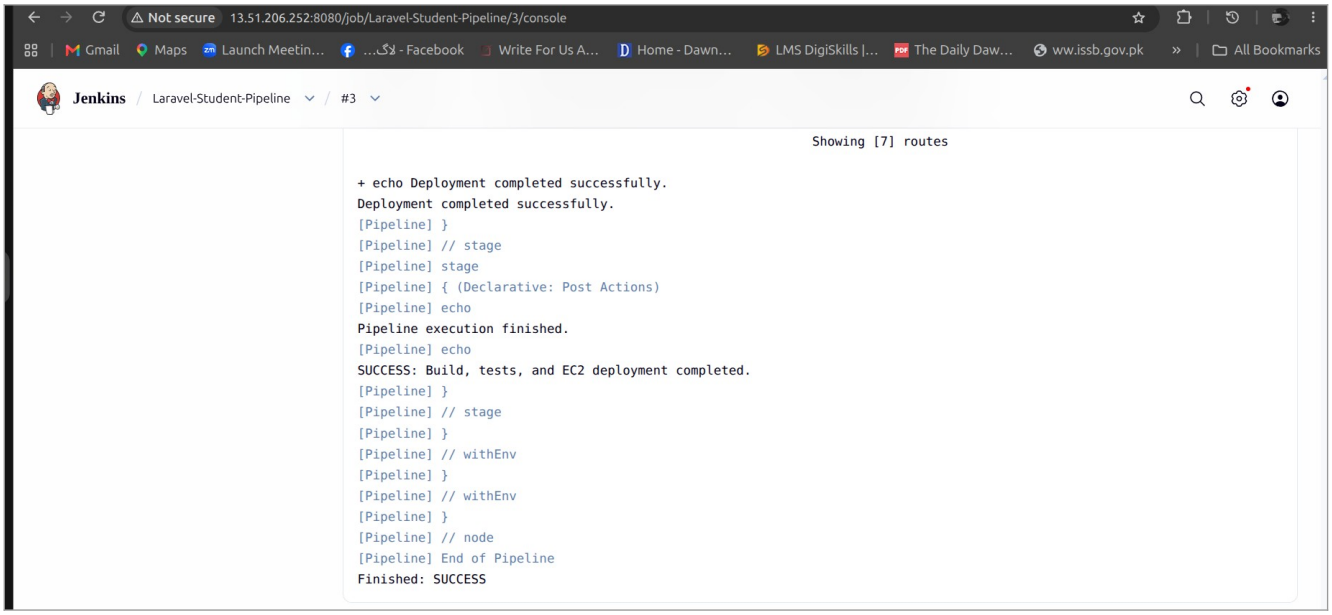


Figure 77: *Console output of pipeline build #3 ending with Finished: SUCCESS.*

The Changes page of build #3 lists the single change that triggered it: commit `d2738c2` (“Update Jenkins pipeline deployment”), which modified the `Jenkinsfile`.

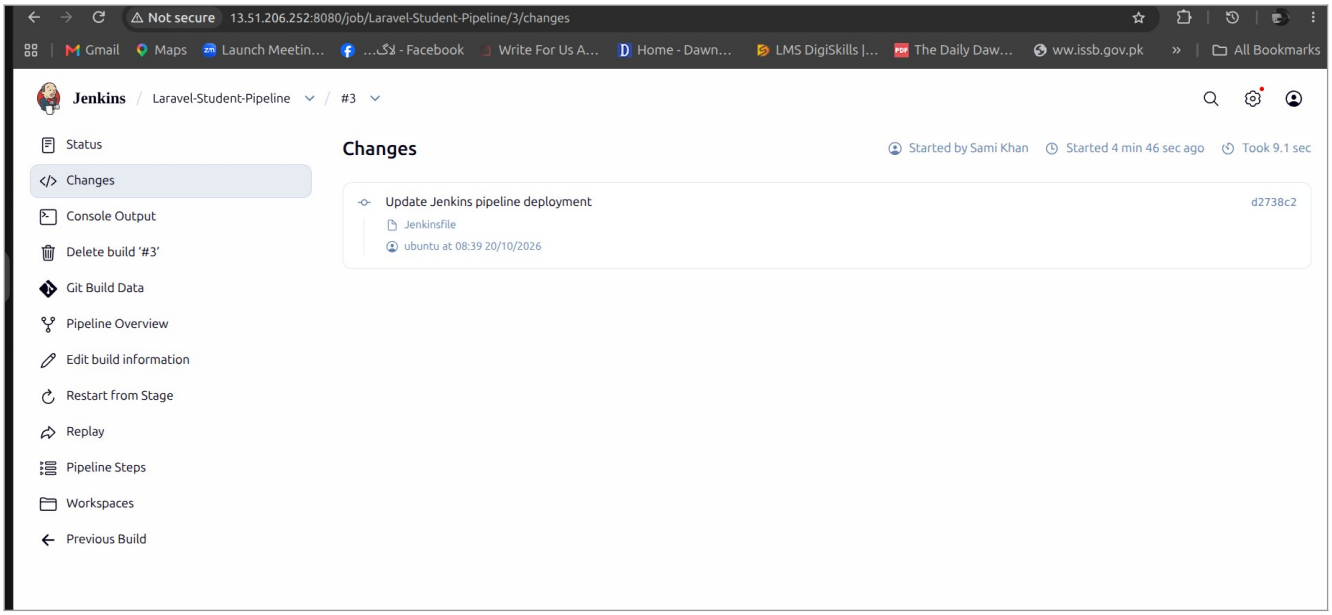


Figure 78: *Changes page of pipeline build #3 showing the updated Jenkinsfile.*

10. Publishing the Application with Nginx

10.1 Configuring the Nginx Site

Until now the application had only been reachable on port 8001. To serve it from the standard HTTP port, I created an Nginx site file named `student-management` in `/etc/nginx/sites-available/` using `nano`. The file in the screenshot declares `listen 80 default_server`, `server_name _`, and sets the web root to `/var/www/student-management/public`, which is Laravel’s public folder. The `index` directive lists

`index.php` and `index.html`, and `location /` uses `try_files $uri $uri/ /index.php?$query_string` so that every request is passed to Laravel's front controller.

```
 GNU nano 8.7.1     /etc/nginx/sites-available/student-management *
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    root /var/www/student-management/public;
    index index.php index.html;

    charset utf-8;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico {
        access_log off;
        log_not_found off;

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Figure 79: *Nginx site configuration for the student-management application.*

10.2 Opening Port 80 in the Security Group

For visitors outside the server to reach Nginx, the Security Group must allow HTTP traffic. I modified its inbound rules in the AWS console. The green banner confirms that the rules of the security group `launch-wizard-6` were modified successfully, and the group now lists four inbound rules: SSH (22), **HTTP (80)**, Custom TCP **8080** for Jenkins and Custom TCP **8001** for the Laravel application.

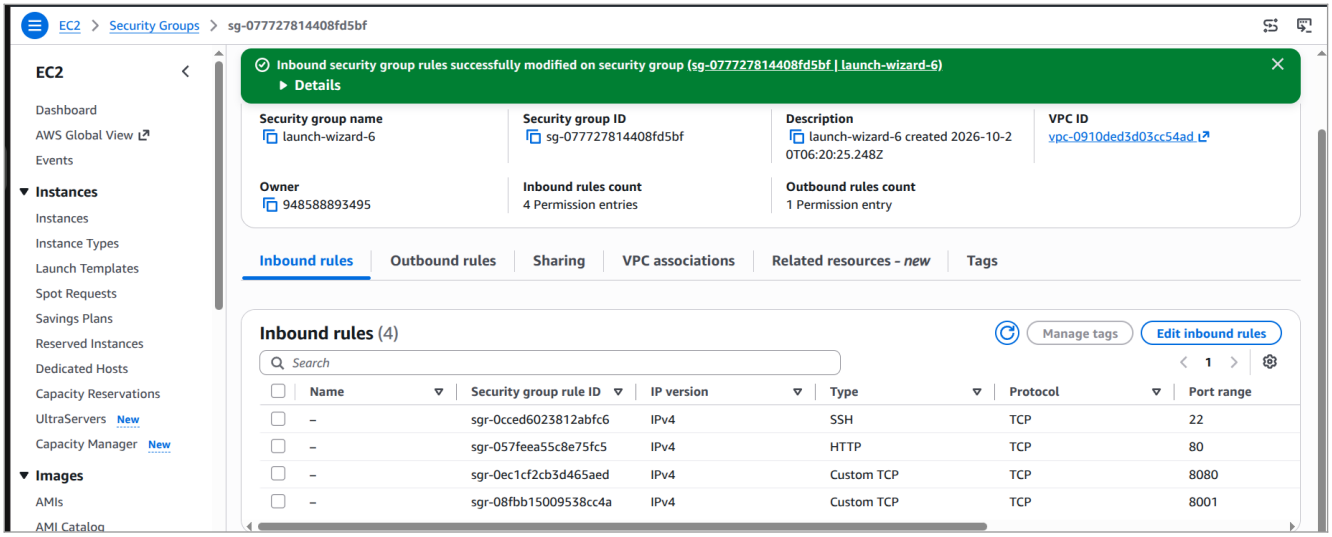


Figure 80: *Security Group launch-wizard-6 after adding the HTTP rule on port 80.*

10.3 Testing the Public Website

Finally, I opened `http://13.51.206.252/students` in the browser, this time without a port number. The Student Management System loads through Nginx and lists both students, `SP23-BSE-092` (Muhammad Sami)

and `SP23-BSE-039` (Sami Khan), each with View, Edit and Delete buttons. This shows that the Laravel application is deployed and reachable from the public internet.

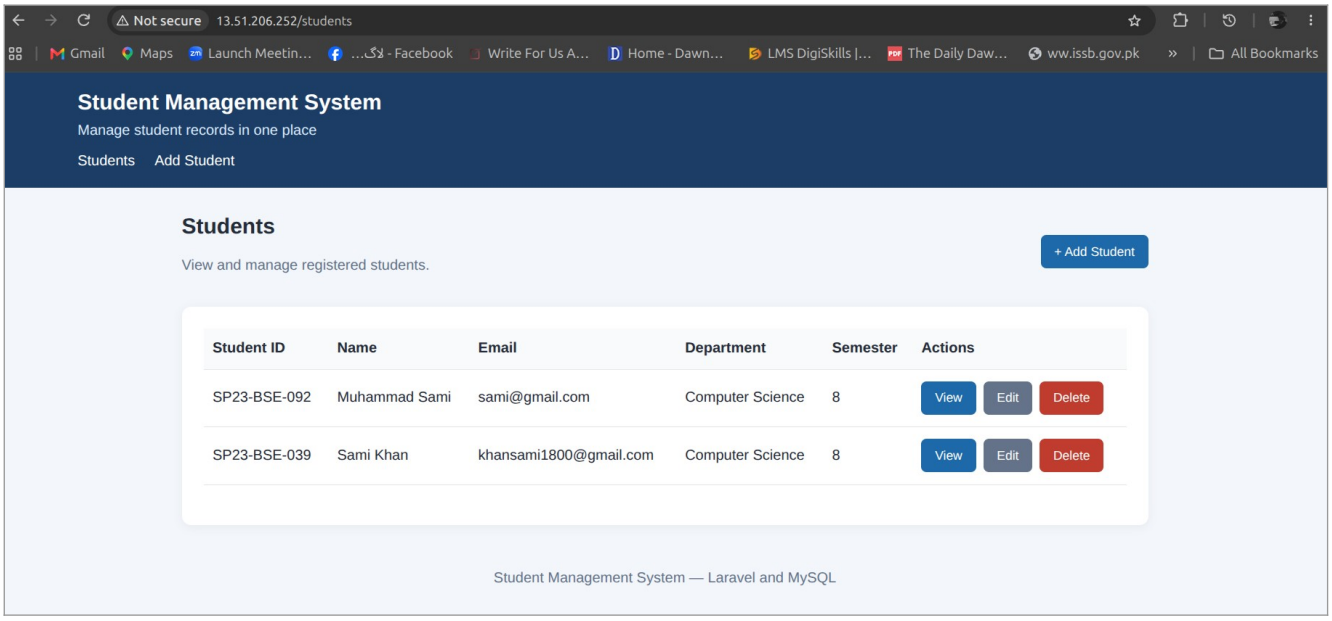


Figure 81: *The Student Management System opened through Nginx at http://13.51.206.252/students.*

I also checked the EC2 console once more. The instance `JenkinsLabTask` (`i-01c3abe1ab17178b7`, type c7i-flex.large) is **Running** and passes 3/3 status checks, with the public DNS name `ec2-13-51-206-252.eu-north-1.compute.amazonaws.com`.

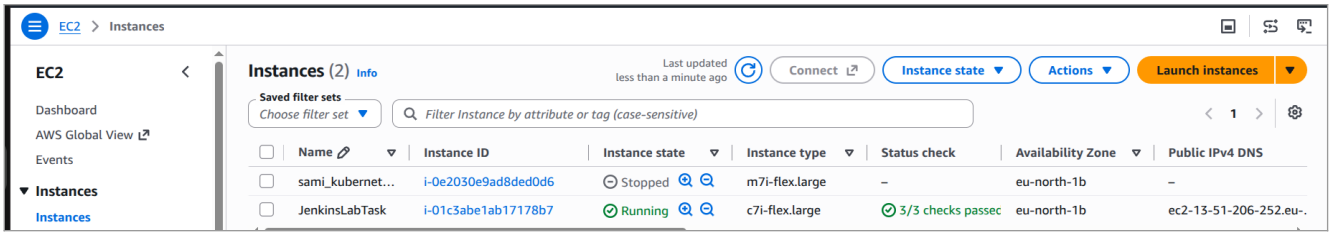


Figure 82: *EC2 instances list showing JenkinsLabTask running with 3/3 status checks.*

11. Conclusion

In this lab I prepared an Ubuntu EC2 server, installed Jenkins, and built a Laravel Student Management System with a MySQL database and full student CRUD pages. I pushed the project to GitHub and automated it in Jenkins, first with three chained Freestyle jobs (build, test and deployment verification) and then with a Pipeline defined in a `Jenkinsfile` whose four stages (Checkout, Build, Test and Deploy to EC2) completed successfully in build #3. I then configured Nginx and the AWS Security Group so that the application is reachable at `http://13.51.206.252/students`.

The lab showed how version control, automated tests and a deployment pipeline reduce manual work and the risk of mistakes. It also showed that infrastructure details such as Security Group rules and file permissions matter as much as the code itself.